

**Controller Design and Description (2022 Controller Revision)****12/6/2022*****Rev. 0******ABSTRACT:***

This document is intended to give MUPP users a solid understanding of the design and structure of the EPKS Control Builder backbone of the 2022 Controller Revision. Included in this document is:

- The design of the final control structure.
- The overall flow of information between the four control modules that form each controller.
- A granular description of the four control modules that form each controller.

Contents

List of Tables	3
List of Figures	3
Document Control and Revision History.....	5
Definitions.....	6
1.0 Purpose	7
2.0 Scope.....	7
3.0 Controller Overview.....	8
5.0 Control Module Implementation.....	10
5.1 Overall Flow of Information	10
5.2 Control Module Overview.....	11
5.2.1 PID_PvSp_ARRAY-R1A.....	11
5.2.2 PID _n _AutVar_SELECT-R0A.....	23
5.2.3 PID _n _PV_SELECT-R1A	34
5.2.4 Final Control Element CM	55

List of Tables

Table 1: Document Control and Revision History	5
--	---

List of Figures

Figure 1: Block diagram representation of final controller design. Dependent on program logic: Flow from FF controller (A) can be made zero, thus disabling the FF controller. Flow can be transferred between paths B and C, toggling the outer-loop control between EPKS and OPC based. Flow can be transferred between paths D and E, including or excluding the inner-loop controller.	8
Figure 2: Diagram illustrating the flow of information between EPKS control modules, a unique version of 'PIDn_AutVar_SELECT', 'PIDn_PV_SELECT', and 'Manipulated Element Controller' CMs exist for each unique controller. 'PID_PvSp_ARRAY' is a single CM that interacts with all other controllers' CMs.	11
Figure 3: PID_PvSp_ARRAY-R1A CM, Page 1.....	12
Figure 4: PID_PvSp_ARRAY-R1A CM, Page 2.....	13
Figure 5: PID_PvSp_ARRAY-R1A CM, Page 3.....	14
Figure 6: PID_PvSp_ARRAY-R1A CM, Page 4.....	15
Figure 7: PID_PvSp_ARRAY-R1A CM, Page 5.....	16
Figure 8: PID_PvSp_ARRAY-R1A CM, Page 6.....	17
Figure 9: PID_PvSp_ARRAY-R1A CM, Page 7.....	18
Figure 10: PID_PvSp_ARRAY-R1A CM, Page 8.	19
Figure 11: PID_PvSp_ARRAY-R1A CM, Page 9.	20
Figure 12: PID_PvSp_ARRAY-R1A CM, Page 10.	21
Figure 13: PID_PvSp_ARRAY-R1A CM, Page 13.	22
Figure 14: PIDn_AutVar_SELECT-R0A, Page 1.....	24
Figure 15: PIDn_AutVar_SELECT-R0A, Page 2.....	25
Figure 16: PIDn_AutVar_SELECT-R0A, Page 5.....	26
Figure 17: PIDn_AutVar_SELECT-R0A, Page 6.....	27
Figure 18: PIDn_AutVar_SELECT-R0A, Page 7.....	28
Figure 19: PIDn_AutVar_SELECT-R0A, Page 8.....	29
Figure 20: PIDn_AutVar_SELECT-R0A, Page 9.....	30
Figure 21: PIDn_AutVar_SELECT-R0A, Page 10.....	31
Figure 22: PIDn_AutVar_SELECT-R0A, Page 11.....	32
Figure 23: PIDn_AutVar_SELECT-R0A, Page 12.....	33
Figure 24: PIDn_PV_SELECT-R1A, Page 1.....	35

Figure 25: PIDn_PV_SELECT-R1A, Page 2.....	36
Figure 26: PIDn_PV_SELECT-R1A, Page 3.....	37
Figure 27: PIDn_PV_SELECT-R1A, Page 4.....	38
Figure 28: PIDn_PV_SELECT-R1A, Page 5.....	39
Figure 29: PIDn_PV_SELECT-R1A, Page 6.....	40
Figure 30: PIDn_PV_SELECT-R1A, Page 7.....	41
Figure 31: PIDn_PV_SELECT-R1A, Page 8.....	42
Figure 32: PIDn_PV_SELECT-R1A, Page 9.....	43
Figure 33: PIDn_PV_SELECT-R1A, Page 10.....	44
Figure 34: PIDn_PV_SELECT-R1A, Page 11.....	45
Figure 35: PIDn_PV_SELECT-R1A, Page 12.....	46
Figure 36: PIDn_PV_SELECT-R1A, Page 13.....	47
Figure 37: PIDn_PV_SELECT-R1A, Page 14.....	48
Figure 38: PIDn_PV_SELECT-R1A, Page 15.....	49
Figure 39: PIDn_PV_SELECT-R1A, Page 16.....	50
Figure 40: PIDn_PV_SELECT-R1A, Page 17.....	51
Figure 41: PIDn_PV_SELECT-R1A, Page 18.....	52
Figure 42: PIDn_PV_SELECT-R1A, Page 19.....	53
Figure 43: PIDn_PV_SELECT-R1A, Page 20.....	54
Figure 44: Final Control Element CM (FCV_570-R1A), Page 1.....	56
Figure 45: Final Control Element CM (FCV_570-R1A), Page 2.....	57
Figure 46: Final Control Element CM (FCV_570-R1A), Page 3.....	58
Figure 47: Final Control Element CM (FCV_570-R1A), Page 4.....	59
Figure 48: Final Control Element CM (FCV_570-R1A), Page 5.....	60
Figure 49: Final Control Element CM (FCV_570-R1A), Page 6.....	61
Figure 50: Final Control Element CM (FCV_570-R1A), Page 7.....	62
Figure 51: Final Control Element CM (FCV_570-R1A), Page 8.....	63
Figure 52: Final Control Element CM (FCV_570-R1A), Page 9.....	64
Figure 53: Final Control Element CM (FCV_570-R1A), Page 10.....	65
Figure 54: Final Control Element CM (FDP_REF_521-R1A), Page 10.....	66

Document Control and Revision History

Table 1: Document Control and Revision History

Revision	Date	Author(s)	Reviewed by	Comments/Details
0	2022	Nicholas O’Keeffe		

Definitions

CM	Control Module
DCS	Distributed Control System
DV	Disturbance Variable
Enum	Enumeration
EPKS	Experion Process Knowledge System
FB	Feedback
FBFF	Feedback augmented Feedforward
FCV	Flow Control Valve
FF	Feedforward
HMI	Human Machine Interface
MUPP	Murdoch University Pilot Plant
MV	Manipulated Variable
OPC	Open Platform Communication
PID	Proportional, Integral, Derivative
PV	Process Variable
SP	Set Point

1.0 Purpose

This document is intended to give users and prospective upgraders of the MUPP the knowledge required to understand the intention, structure, features, and principles of the controller designed for the 2022 MUPP controller revision.

2.0 Scope

This document covers all aspects of the EPKS Control Builder side of the 2022 controller revision, including:

- A simplified overview of the fundamental control structure.
- The interaction and flow of information between the four control modules that make up each the controllers in the revised controller.
- A thorough break down of the function and purpose of the following CMs:
 - **PID_PvSp_ARRAY-R1A**
 - **PID n _AutVar_SELECT-R0A**
 - **PID n _PV_SELECT-R1A**
 - **Manipulated Element Controller**

3.0 Controller Overview

The controller requirements implemented in the 2022 MUPP controller revision included coverage of the foundational control systems that would encompass the control topologies covered by future Murdoch University Industrial Control and Automation Engineering students. These control strategies are:

- Single-loop feedback PID
- Cascaded-loop feedback PID
- Feedforward

As well as the ability to switch between the above control strategies, the implemented controller should be able to switch between these and combinations of these control systems dynamically. Although future students will not be covering control systems beyond these in the undergraduate degree, it is desired to include the ability to, using some method, be able to implement and experiment with control strategies outside the scope of the EPKS system.

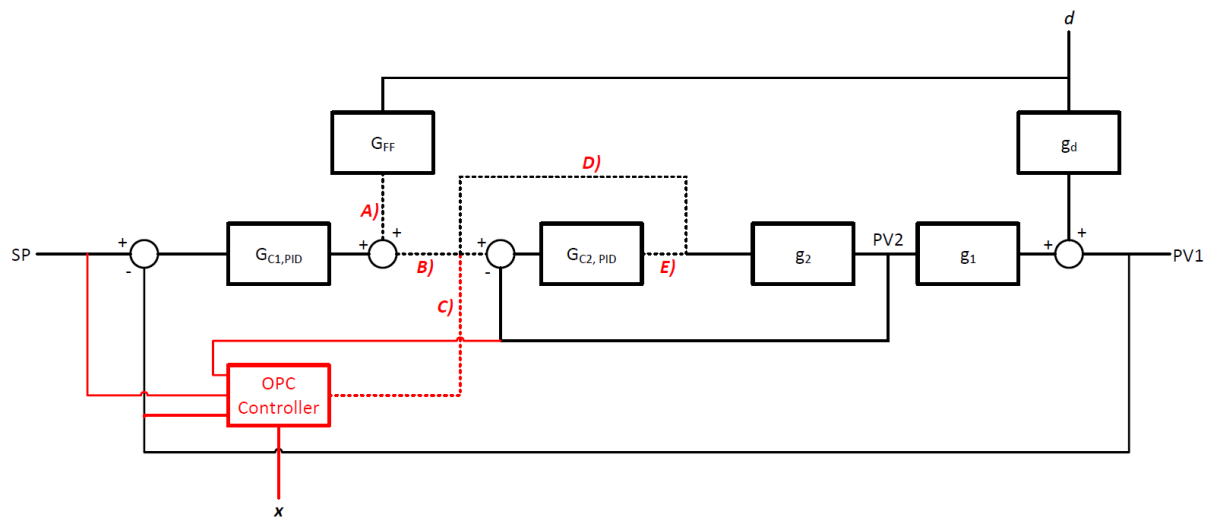


Figure 1: Block diagram representation of final controller design. Dependent on program logic: Flow from FF controller (A) can be made zero, thus disabling the FF controller. Flow can be transferred between paths B and C, toggling the outer-loop control between EPKS and OPC based. Flow can be transferred between paths D and E, including or excluding the inner-loop controller.

The nature of EPKS is such that the dynamic switching between vastly different control strategies is not easily implemented. As EPKS is intended to be used to develop a final product (*an industrial DCS*), and an industrial process control system is, in practice, developed before the control system it is housed in, there is not usually a requirement for dynamically changeable control systems. The solution to this challenge is given in Figure 1, which shows the block diagram representation of the final controller design. The design is, in essence, a cascade feedback augmented feedforward controller; however, depending on program

logic (*not included in Figure 1*), input into the physical system is routed from different locations in the control loop.

Depending on program logic:

- The feedforward output (Figure 1, A) can be routed into or removed from the loop.
- The output of the outer loop can be taken from the EPKS outer-loop PID+FF controller (Figure 1, B) or the OPC variable (Figure 1, C).
- The process flow can bypass the inner-loop PID controller (Figure 1, D) or flow through it (Figure 1, E).

By designing the controller in this way, a highly configurable controller can be implemented in the EPKS environment.

5.0 Control Module Implementation

The following sections will describe how the controller described in section 3.0 *Controller Overview* was actualized in the EPKS control builder. First, the purpose of each control module and how they fit together will be clarified before diving deeper into the function and rationale of the modules themselves.

5.1 Overall Flow of Information

The control system is broken up into four elements (*Figure 2*):

1. **PID_PvSp_ARRAY-R1A** – A single CM that houses the arrays required for all controllers; these arrays include the selected variables (*SP, PV1, PV2, DV*) for each controller (*Stored as an integer number*) and the actual value of all MUPP variables, e.g., level-, temperature-, density-transmitters, etc.
2. **PIDn_AutVar_SELECT-R0A** – A CM for each controller ($n = 1, \dots, 20$), which is responsible for taking the desired variables from EPKS Station and checking if they are in use by another controller before they become the actual selected variables.
3. **PIDn_PV_SELECT-R1A** – A CM for each controller ($n = 1, \dots, 20$), which takes the integers corresponding to the selected variables and finds the actual values corresponding to these selected variables.
4. **Manipulated Element Controller** – The actual controller for the control element; this can be a pump reference speed or flow control valve position CM.

PIDn_AutVar_SELECT-R0A receives the desired PV1 and PV2 variables from EPKS Station; it then compares these integer numbers to the currently selected variables for all controllers stored in **PID_PvSp_ARRAY-R1A**. If any other controllers do not use the desired PV1 and PV2 variables, **PIDn_AutVar_SELECT-R0A** will output these values as the newly selected PV1 and PV2 variables in **PID_PvSp_ARRAY-R1A**.

PID_PvSp_ARRAY-R1A takes the integers corresponding to the selected PV1 and PV2 variables from **PIDn_AutVar_SELECT-R0A** and the selected DV directly from EPKS Station and routes them to the appropriate **PIDn_PV_SELECT-R1A** CM. The arrays of the actual values of PV1, PV2, and DV are also routed. **PIDn_PV_SELECT-R1A** takes the integers corresponding to the selected variables, finds the corresponding actual values in the variable arrays and routes them back to the **PID_PvSp_ARRAY-R1A** CM. The actual variable values are stored in separate arrays in the array index corresponding to the appropriate controller.

Finally, **PID_PvSp_ARRAY-R1A** routes the actual selected variables (*PV1, PV2, and DV*) to the appropriate **Manipulated Element Controller**. The **Manipulated Element Controller** takes these values and desired controller configuration information from the EPKS HMI and outputs the corresponding control action to the final control element.

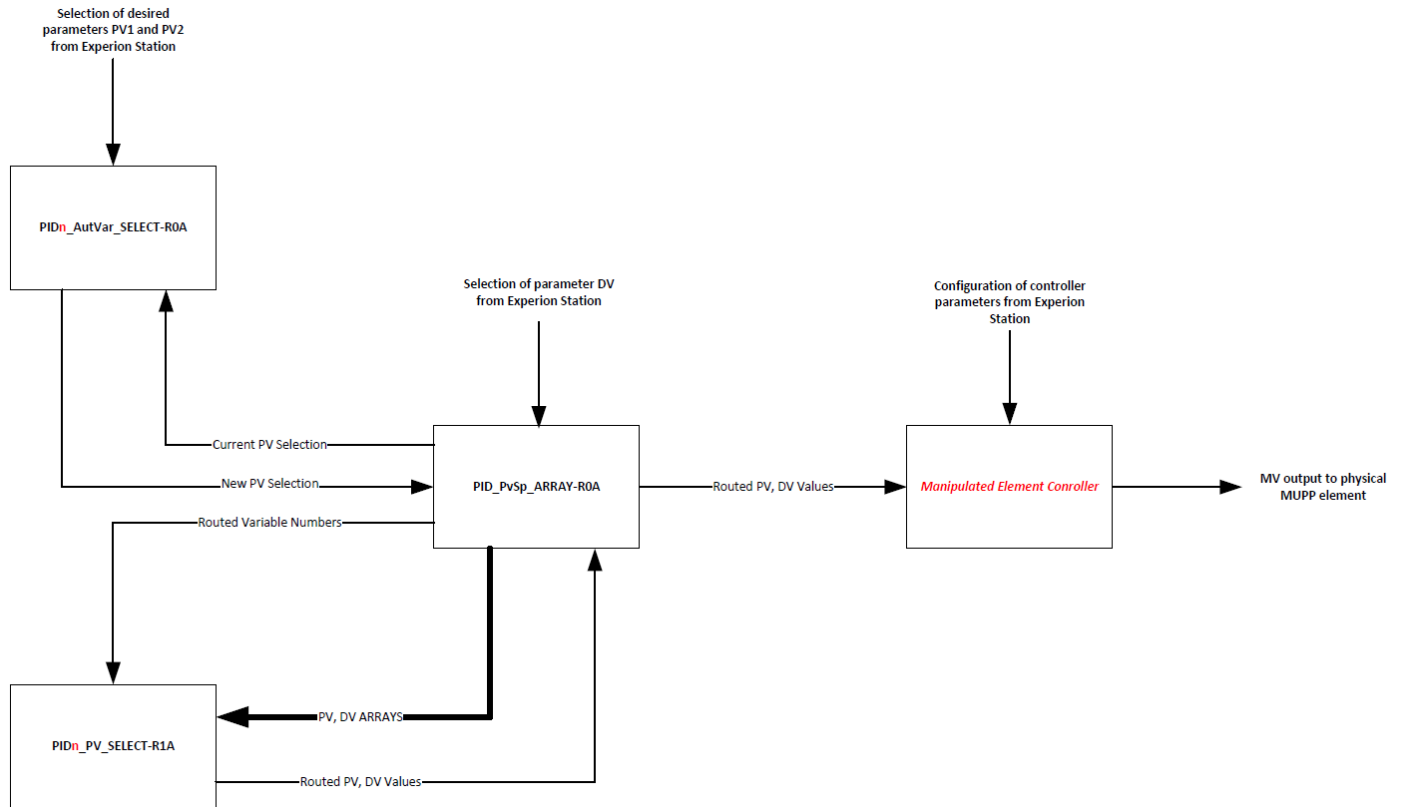


Figure 2: Diagram illustrating the flow of information between EPKS control modules, a unique version of 'PIDn_AutVar_SELECT', 'PIDn_PV_SELECT', and 'Manipulated Element Controller' CMs exist for each unique controller. 'PID_PvSp_ARRAY' is a single CM that interacts with all other controllers' CMs.

5.2 Control Module Overview

The following sections contain explanations for the function of each of the control modules mentioned in section 5.1 *Overall Flow of Information*. For each module, a description of the program flow will be given, followed by figures of the annotated pages of the corresponding CM.

5.2.1 PID_PvSp_ARRAY-R1A

The **PID_PvSp_ARRAY-R1A** CM is adequately explained by viewing Figures 3 to 13. Of note is Figure 13, which shows a CM function not mentioned in the above sections. The **PID_PvSp_ARRAY-R1A** CM takes the integers corresponding to the selected controller variables (PV1, PV2, and DV) and outputs ENUM text corresponding to the selected variable. The purpose of this text is to display in EPKS Station.

	A	B	C	D	E	F	G	H
1	PID_PvSp_ARRAY-R0A Control Module Description:							1
	This control module serves the purpose of:							
2	1- Collecting analog input data from PP sensors. 2- Collecting SP data from EPKS HMI. 3- Storing memory data relating to controller's selected inner- and outerloop PV's, and DV's. 4- Routing actual inner- and outerloop PV, and DV values from PV select CMs to controller CMs.							2
3	KEY FUNCTIONS: TITLE PAGE DOCUMENTATION PAGE 1 Actual AI Input Collection From PP Sensors PAGE 2 Actual SP Input Collection From EPKS HMI PAGE 3 Outer- and Innerloop Selection Parameters From EPKS HMI PAGE 4 Outerloop PV Routing to Controllers PAGE 5 Innerloop PV Routing to Controllers PAGE 6 SP Routing to Controllers PAGE 7 Actual DV Input Collection from PP Sensors PAGE 8 DV Selection Parameters from HMI PAGE 9 DV Routing to Controllers PAGE 10 ENUM casting for HMI PAGE 13-29							3
4								4
5								5
6								6
7	References: AUTHOR: NICHOLAS O'KEEFFE <i>As part of Thesis WU3, 2022</i> REVISION HISTORY: Made with some reference to PID_PV_ARRAY (R0) <i>Written for the PLC5 to C300 Migration.</i>							7
8								8
9	PID_PvSp_ARRAY-R1A communicates with each of the configured controllers: - PIDn_PV_SELECT-R1A - PIDn_AutVar_SELECT-R0A - Final Control Element CM							9
	A	B	C	D	E	F	G	H

Figure 3: PID_PvSp_ARRAY-R1A CM, Page 1.

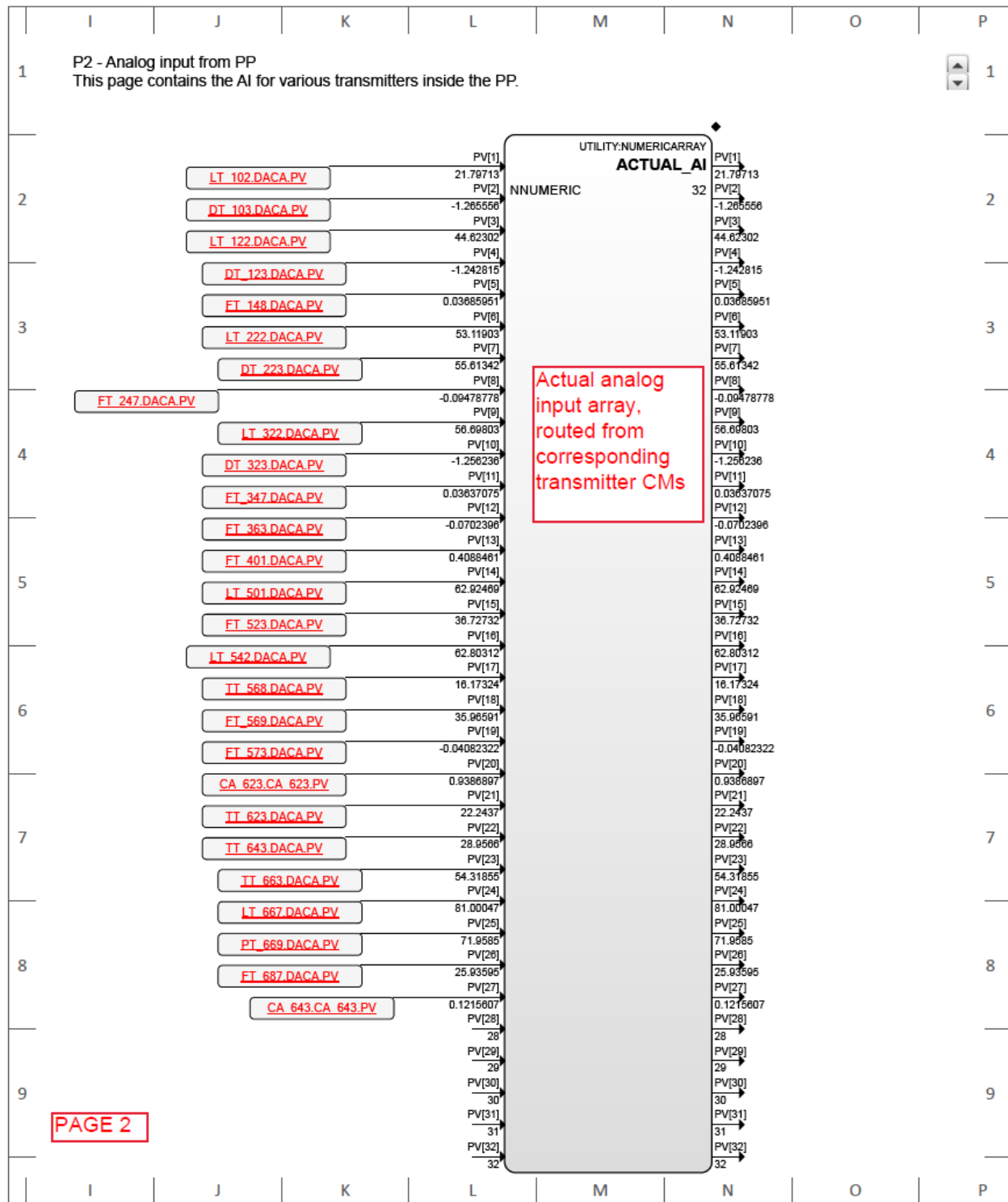


Figure 4: PID_PvSp_ARRAY-R1A CM, Page 2.

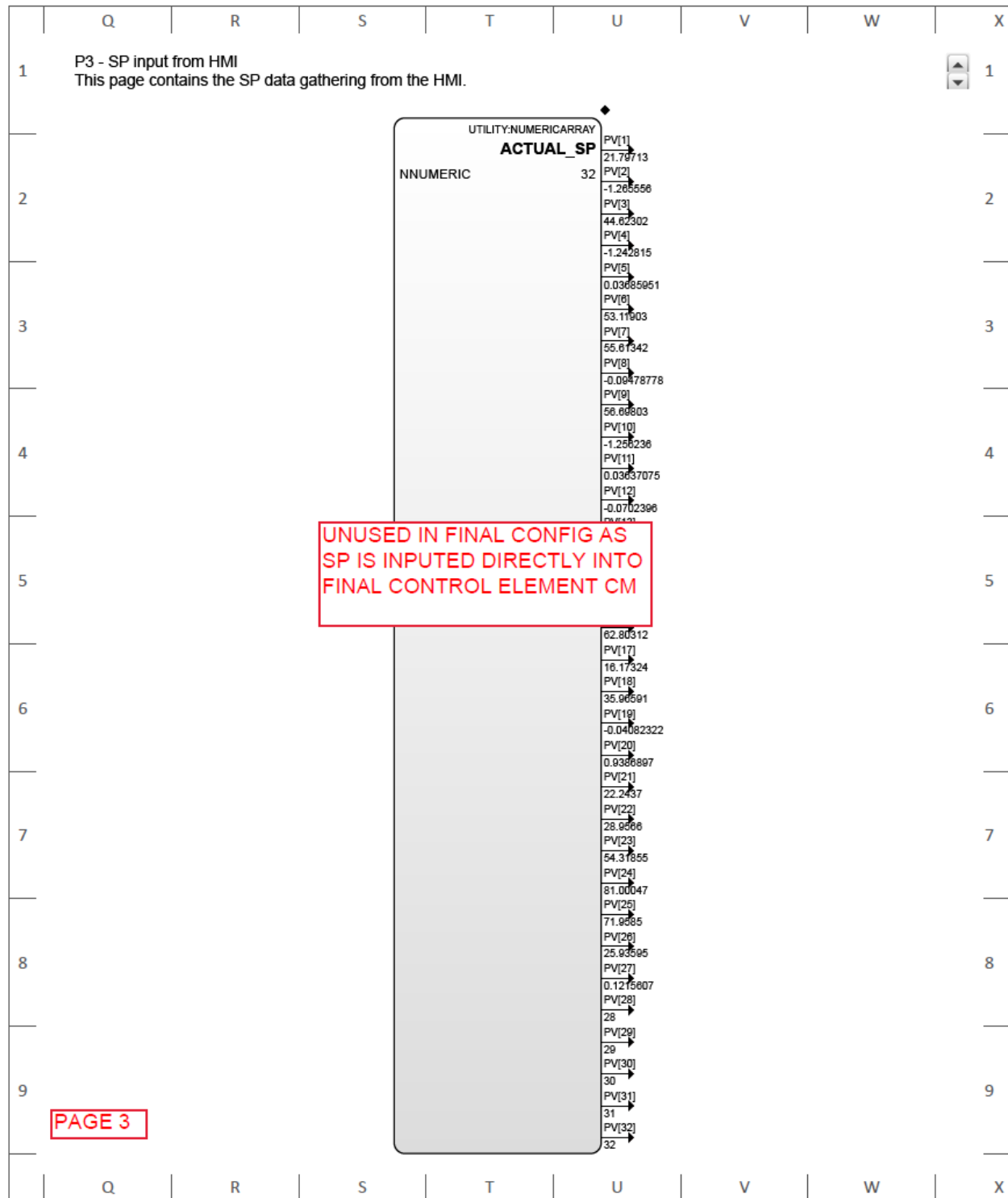


Figure 5: PID_PvSp_ARRAY-R1A CM, Page 3.



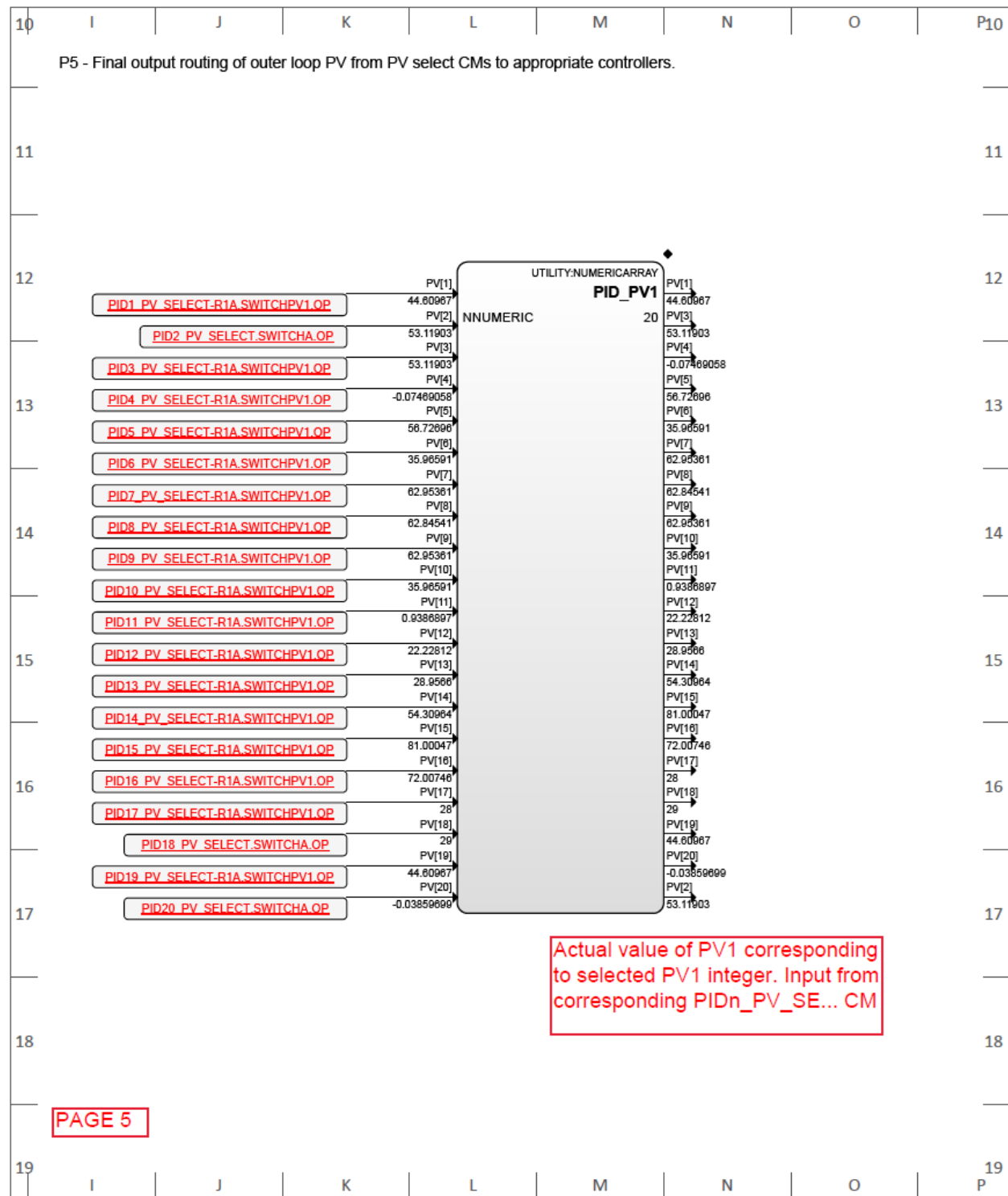


Figure 7: PID_PvSp_ARRAY-R1A CM, Page 5.

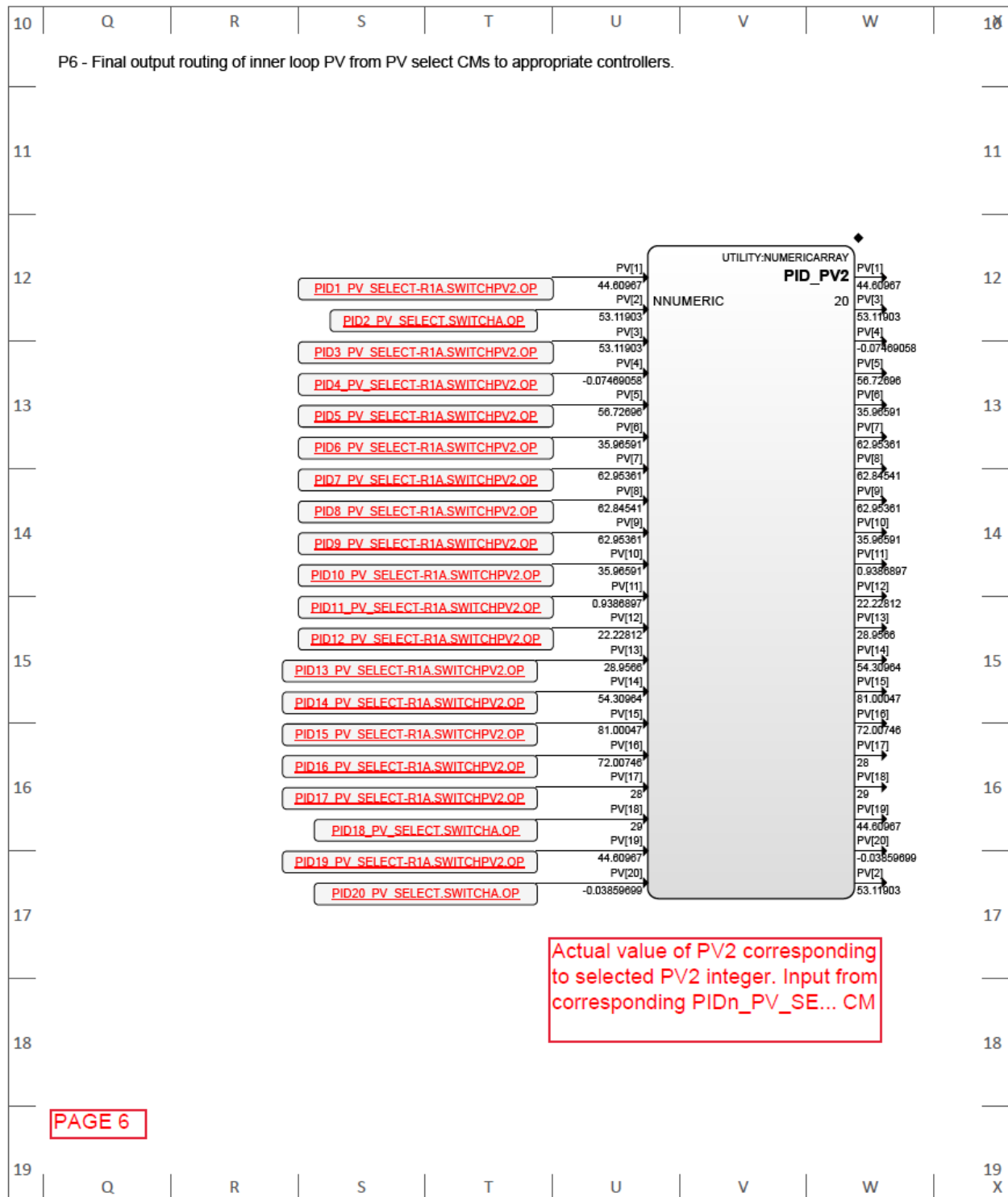


Figure 8: PID_PvSp_ARRAY-R1A CM, Page 6.

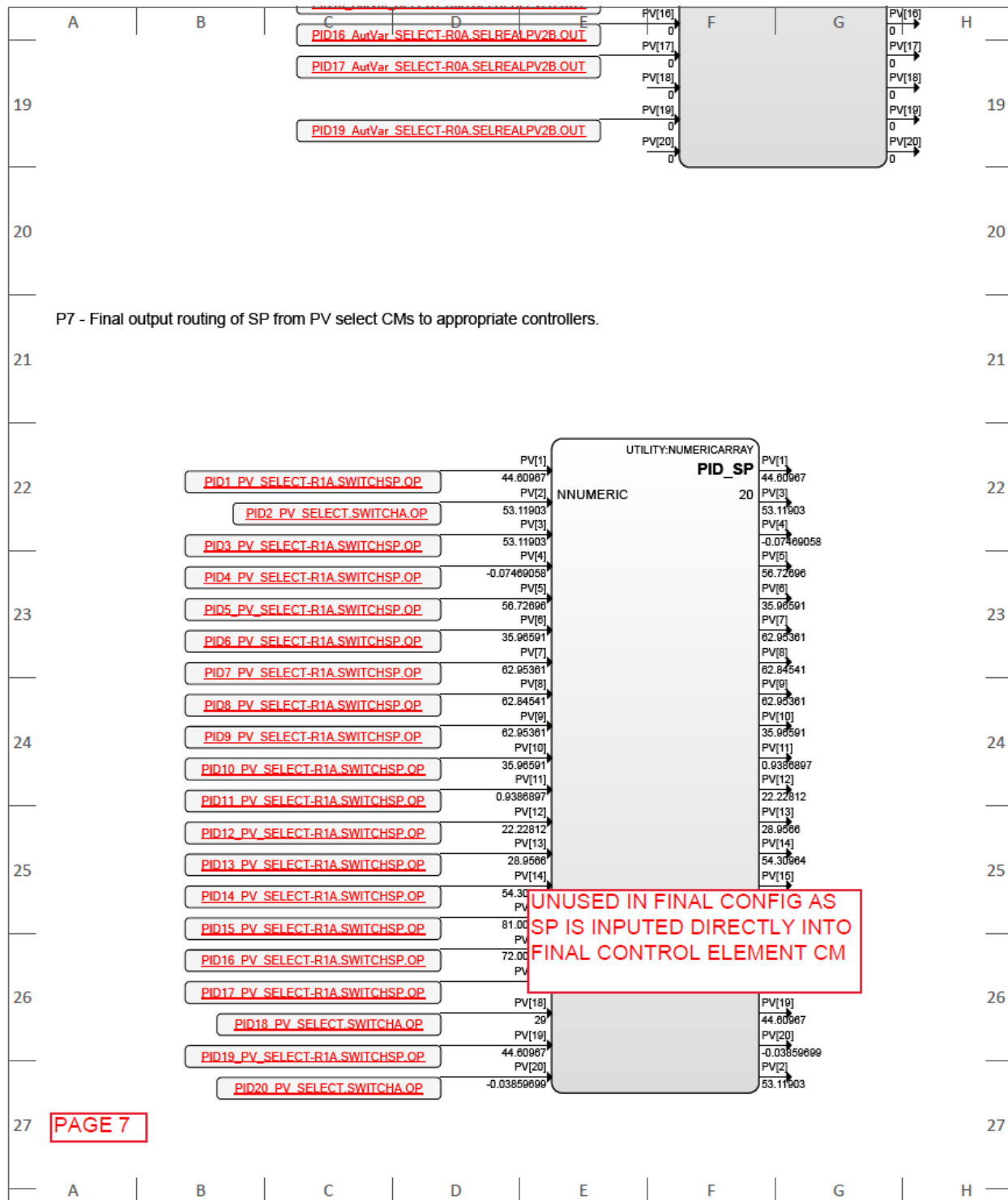


Figure 9: PID_PvSp_ARRAY-R1A CM, Page 7.

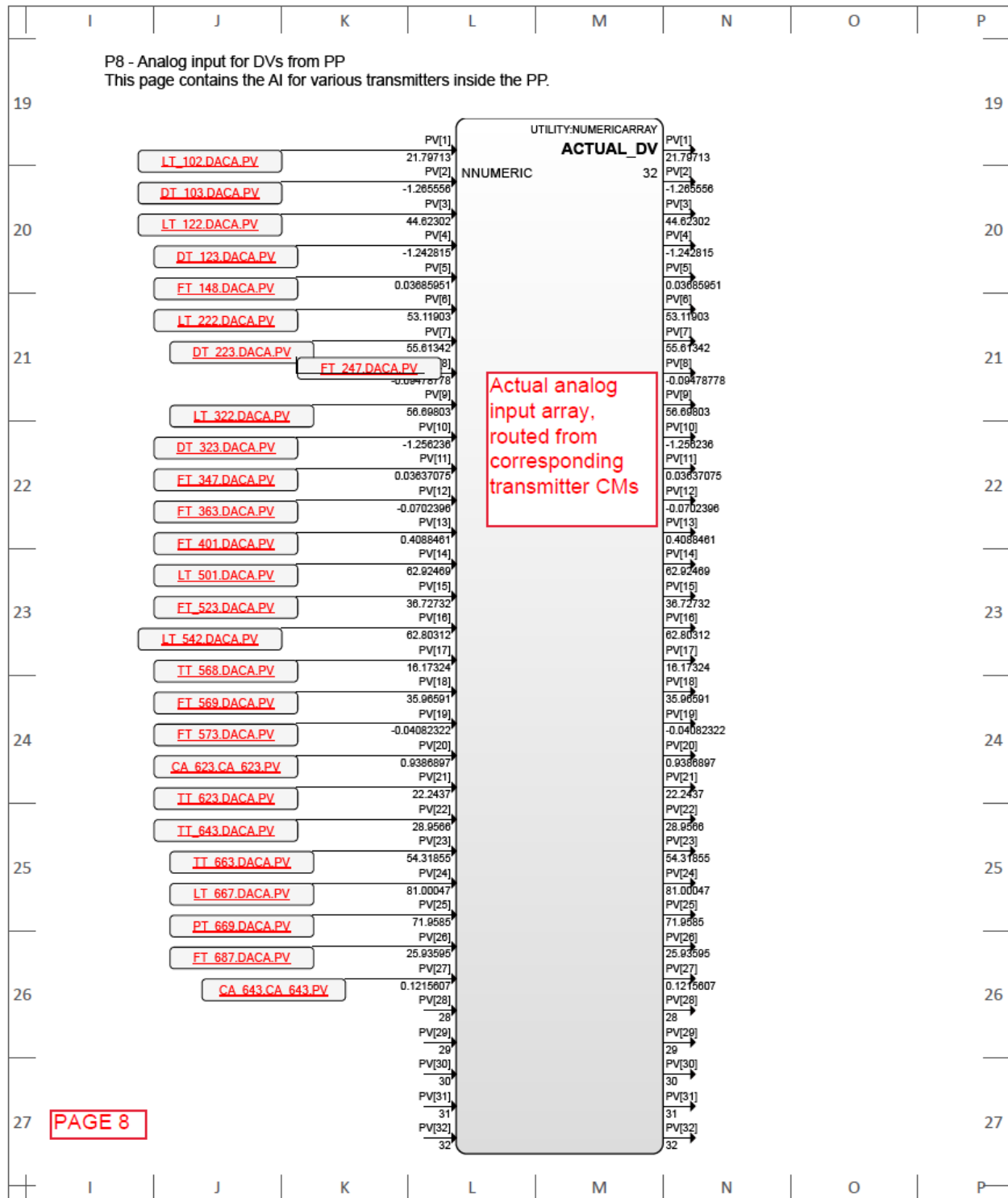


Figure 10: PID_PvSp_ARRAY-R1A CM, Page 8.

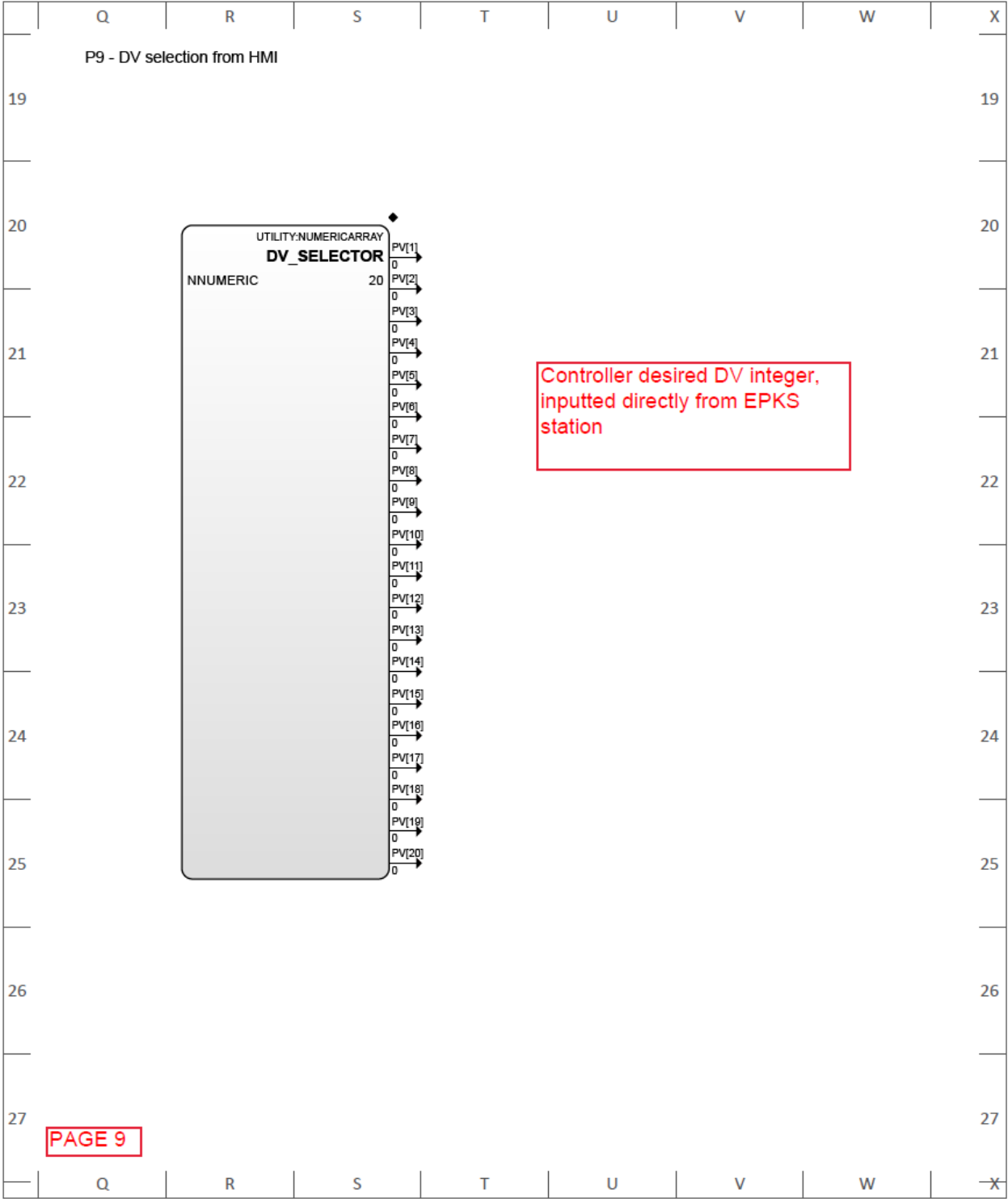


Figure 11: PID_PvSp_ARRAY-R1A CM, Page 9.

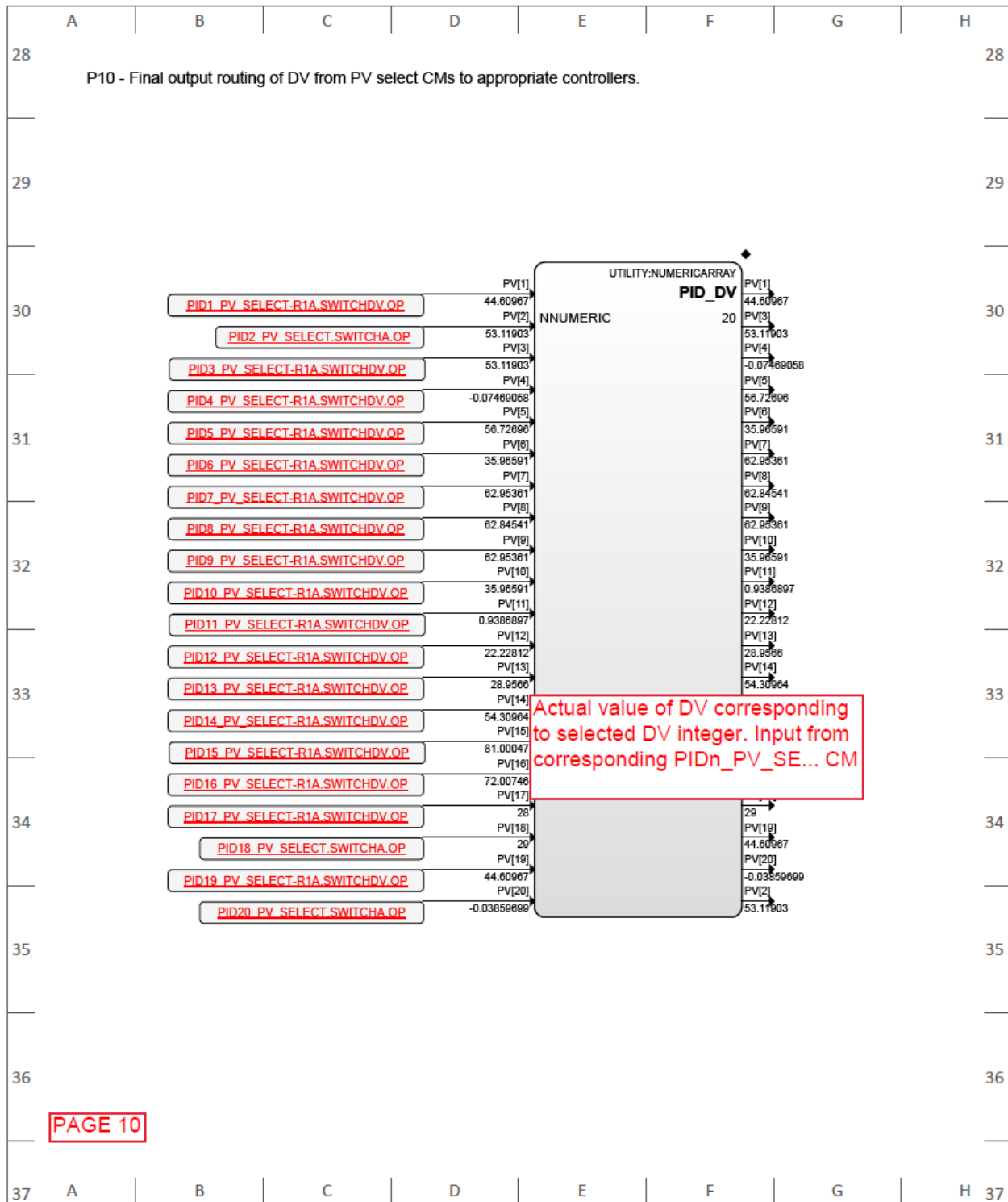


Figure 12: PID_PvSp_ARRAY-R1A CM, Page 10.

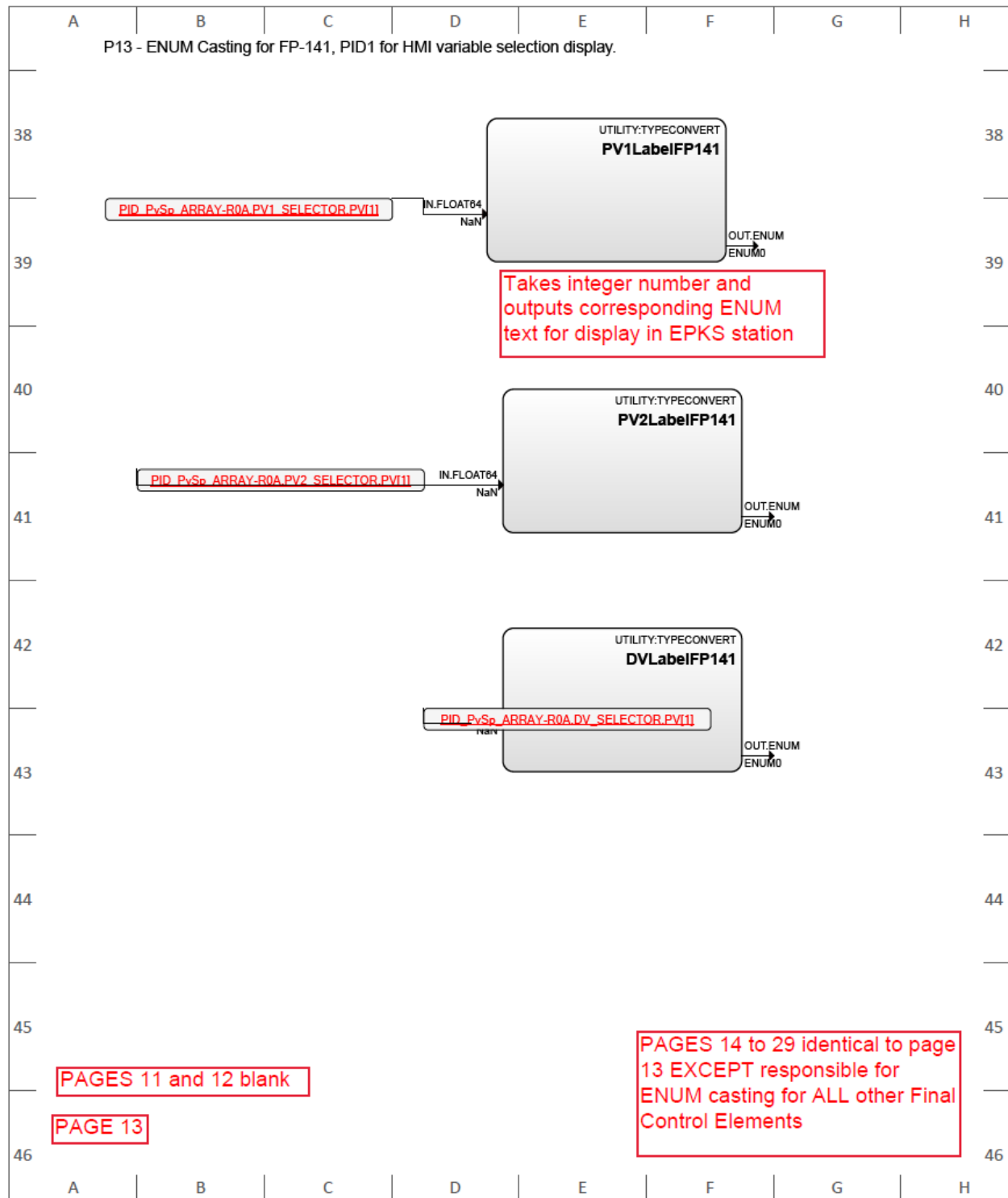


Figure 13: PID_PvSp_ARRAY-R1A CM, Page 13.

5.2.2 PIDn_AutVar_SELECT-R0A

EPKS Station writes the integer corresponding to the desired variable to assign to PV1 (*Figure 16*), which is fed into the 'AUXCALC' function block (*Figure 17*). The 'AUXCALC' function block can house eight logical expressions, which are written as *IF*, *ELSE-IF*, and *ELSE* statements comparing the integer number of the desired variable to the integer value of selected variables in all other controllers. If there is a match, the block will output the integer value '1'; otherwise, the block will output the integer value '0'. **NOTE: EPKS control builder sites a recommended limit of variables per logical expression; as a result, seven logical expressions are used to check all selected variables.**

As the 'AUXCALC' block outputs an integer value, these outputs are then converted into FLAGS (*EPKS equivalent of Boolean*); this is done using an 'EQUALS' function block, which then sends the FLAG to an 'OR' logical function block (*Figure 18*). The 'OR' function block will output a TRUE signal if any logical expressions find a match between the desired variable and the selected variables in any other controller.

The OR function block is routed into an AND function block (*Figure 19*). The second input of the AND function block is the output of another 'EQUALS' function block, which checks if the desired variable is the integer '0', corresponding to "No Variable Selected". Thus, if there is a match in the 'AUXCALC' block, but the new desired value corresponds to the "No Variable Selected" option, the desired value is allowed to proceed.

The 'AND' block outputs to the gate of a 'SELREAL' block, which will route one of two inputs to the output, depending on the gate's position (*Figure 19*). If the output of the 'AND' block is such that the desired variable is in use by another controller, AND the desired variable does NOT correspond to "No Variable Selected", the 'SELREAL' block will output the integer value corresponding to the selected variable. Otherwise, the integer value corresponding to the desired variable will be outputted (*Figure 19*). The output of the 'SELREAL' block is routed to the **PID_PvSp_ARRAY-R1A** CM, thus, becoming the newly selected variable.

Simultaneously, the output of the 'AND' block is routed to the gate of another 'SELREAL' block (*Figure 16*). This block functions identically to the 'SELREAL' block mentioned in the above paragraph, except this block outputs the 'NUMERIC' block containing the desired variable to assign to PV1. Thus, overwriting the desired variable value with the previously selected variable value, IF required.

It is worth noting that the order of operations of these function blocks is critical; these blocks function as a feedback loop, both reading from and writing to the desired variable 'NUMERIC' block. If the order of operations is incorrect, the CM WILL NOT function correctly.

The above process is repeated for the desired variable to assign the PV2 variable in figures 9 to 12.

	A	B	C	D	E	F	G	H
1			PID1_AutVar_SELECT-R0A					1
			Control Module Description.					
			The purpose of this CM is to prevent more than one controller from using PV1 and PV2 values at a time. The primary concern of this is in cascade loops when more than one controller is attempting to reach different setpoints for a given PV.					
2								2
			The strategy for achieving this is having the HMI write "Desired" PV selection values to a temporary variable, this can then be cross checked against the "Actual" values of all other controllers selected PVs. If this comparison finds that the desired PV is not already in use, it will then be moved to the PVn_SELECTED variable in PID_PvSp_ARRAY-R0A CM. Otherwise, the desired PV will be changed to the PVn_SELECTED variable from the PID_PvSp_ARRAY-R0A CM, thus reverting it to its previous value.					
3								3
			KEYFUNCTIONS:					
			TITLE PAGE DOCUMENTATION			PAGE 1		
4			Definition of Constants			PAGE 2		4
			BLANK PAGE			PAGE 3		
			BLANK PAGE			PAGE 4		
5			Definition of desired PV1 from HMI			PAGE 5		5
			Cross checking of Other Controllers Selected PV1s by AUXCALC			PAGE 6		
			Checking for True Outputs from AUXCALC for PV1			PAGE 7		
6			PV1 Select Output Routing Dependent on AUXCALC Findings			PAGE 8		6
			Definition of desired PV2 from HMI			PAGE 9		
			Cross checking of Other Controllers Selected PV2s by AUXCALC			PAGE 10		
7			Checking for True Outputs from AUXCALC for PV2			PAGE 11		7
			PV2 Select Output Routing Dependent on AUXCALC Findings			PAGE 12		
			References:					
8			AUTHOR: NICHOLAS O'KEEFFE As part of Thesis WU3, 2022					8
			REVISION HISTORY: REV0					
9								9
	A	B	C	D	E	F	G	H

PIDn_AutVar_SELECT-R0A exists for each of the twenty configured controllers, only variable names change.

Figure 14: PIDn_AutVar_SELECT-R0A, Page 1.

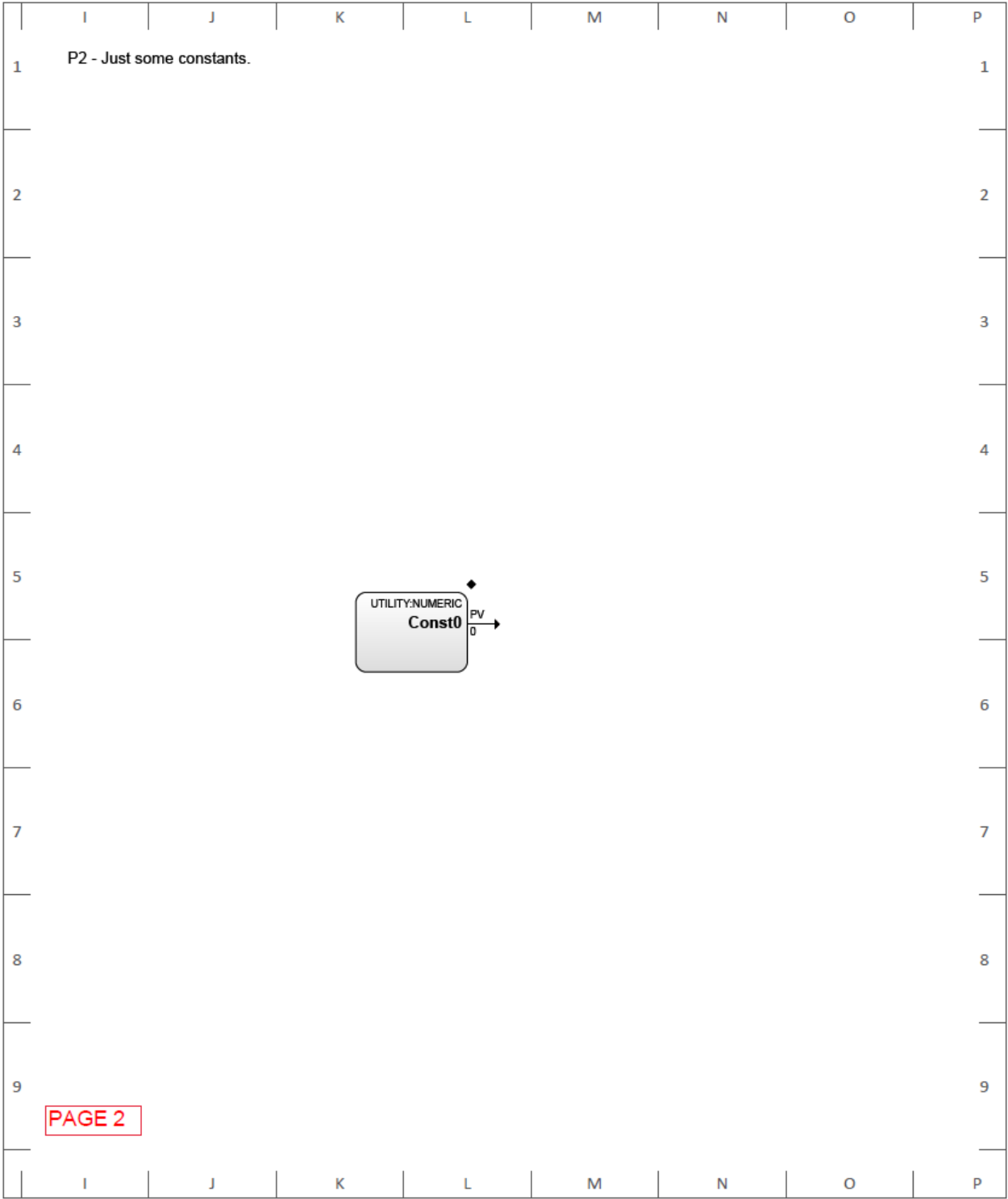


Figure 15: PIDn_AutVar_SELECT-R0A, Page 2.

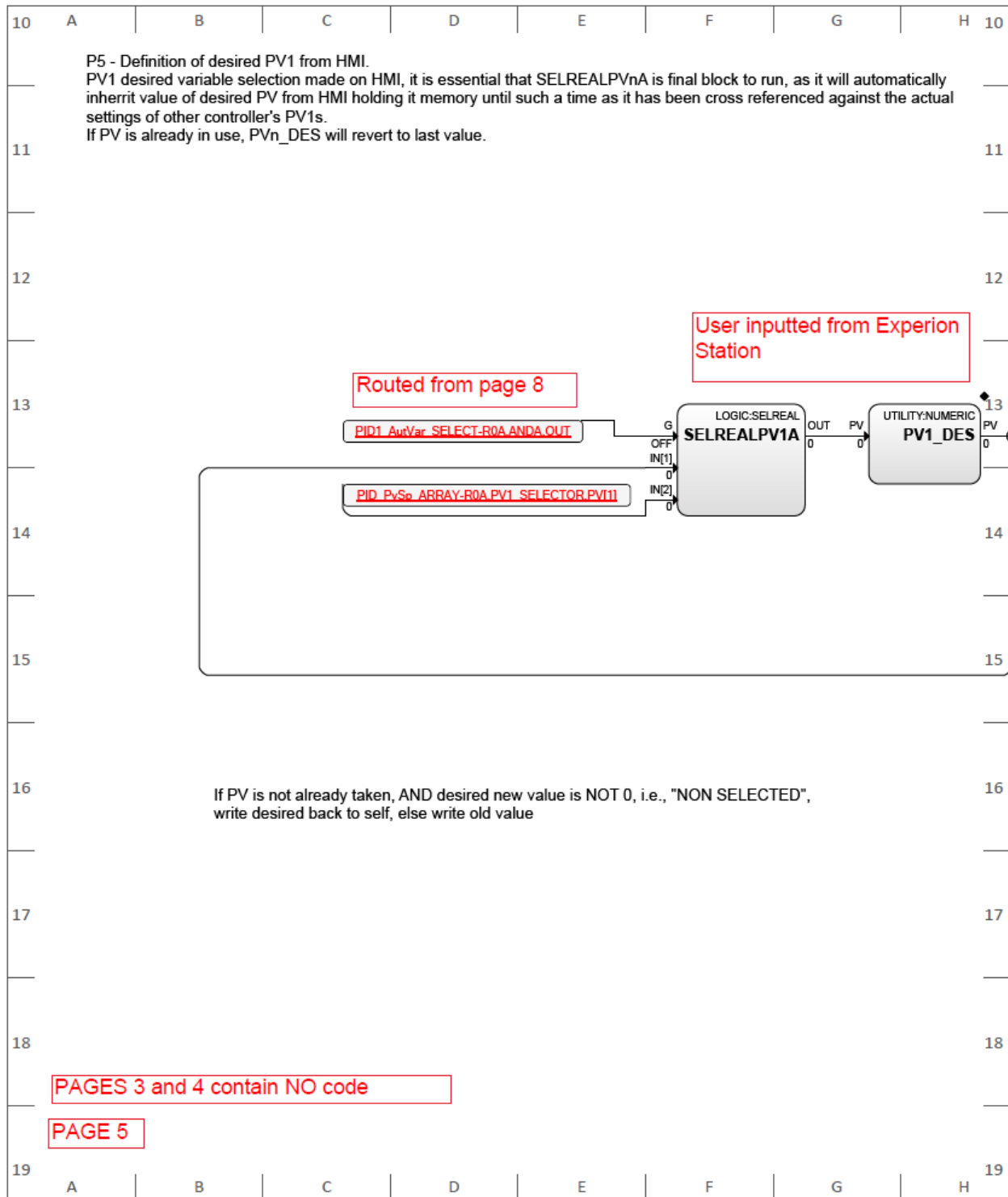


Figure 16: PIDn_AutVar_SELECT-R0A, Page 5.

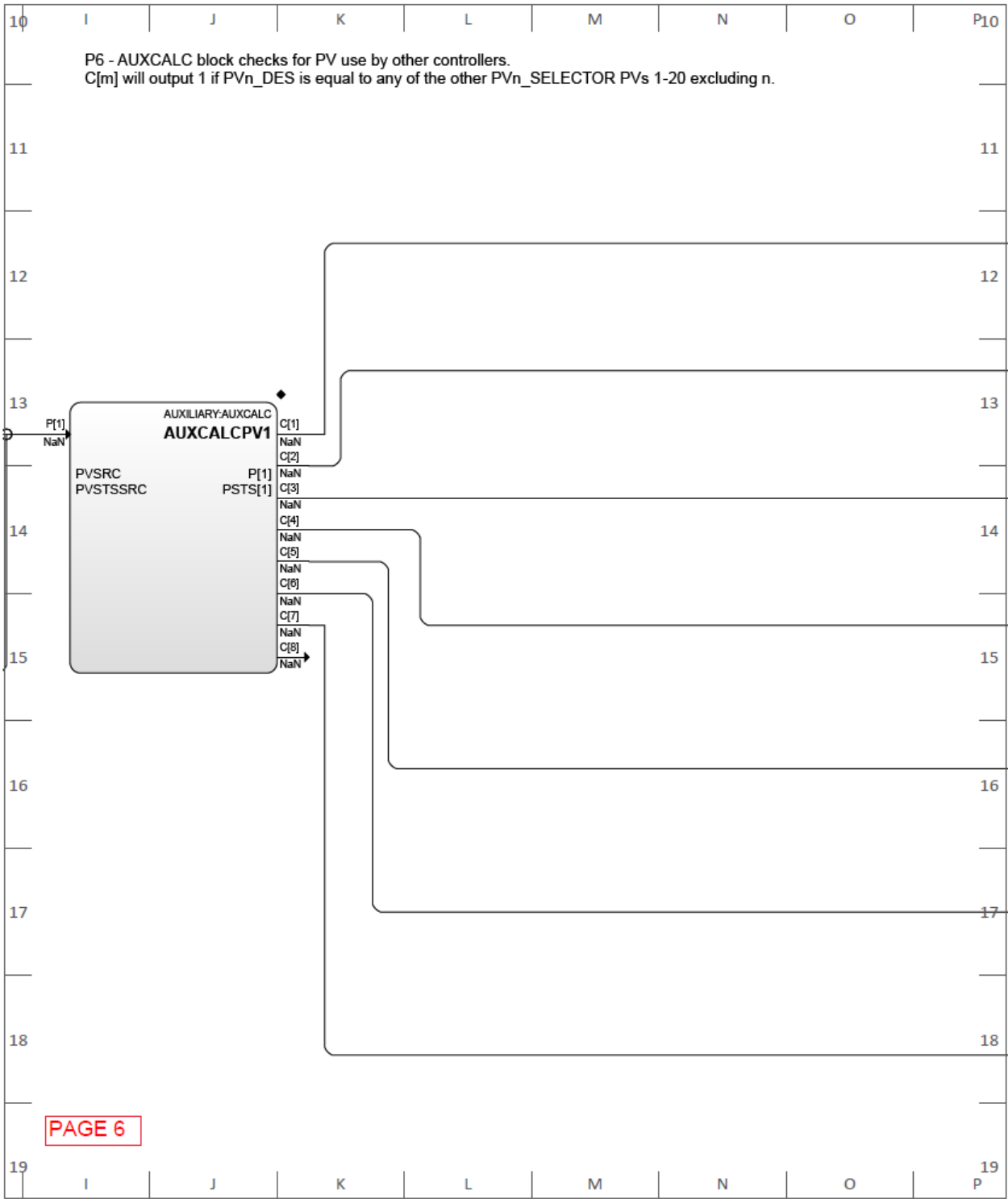
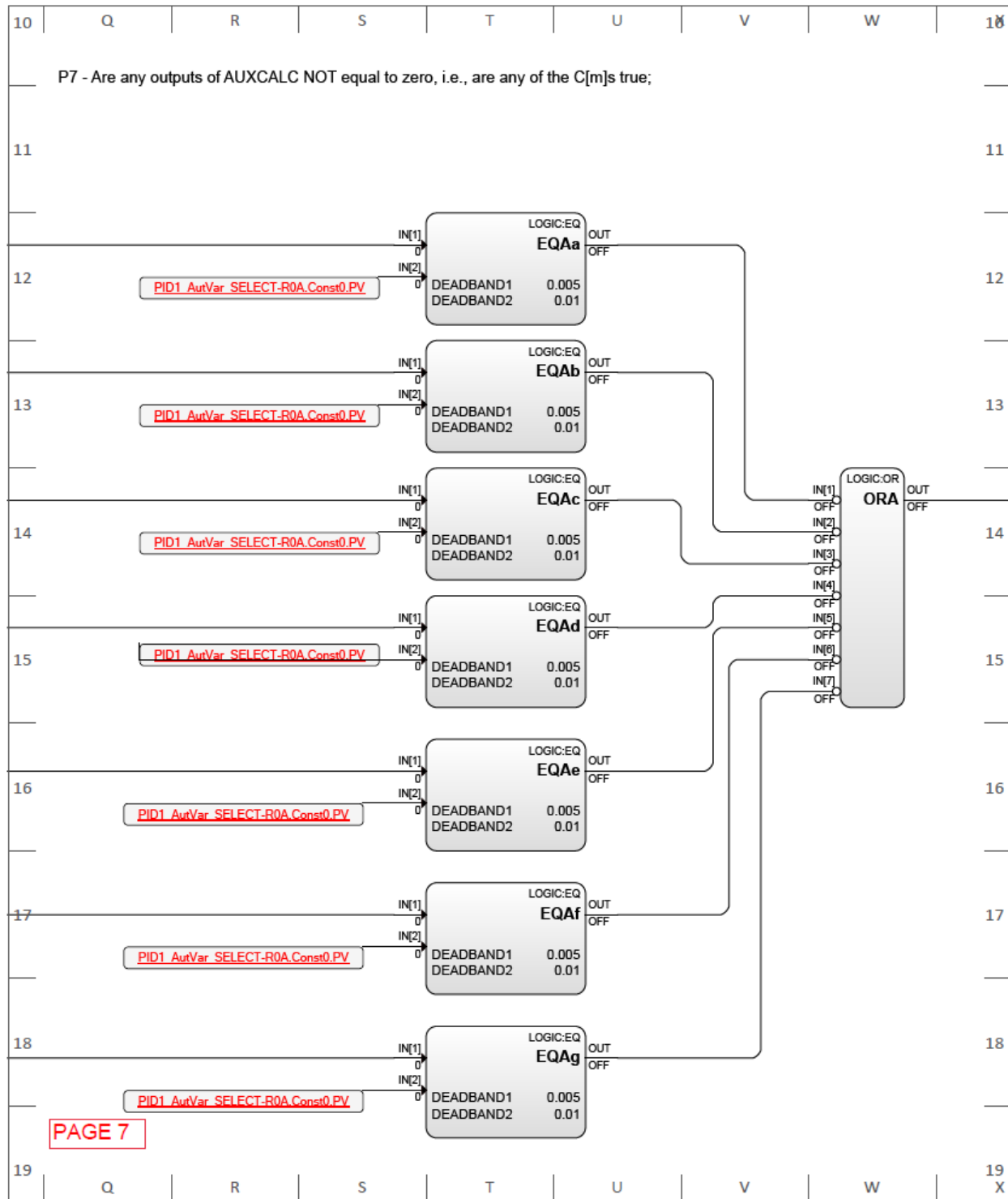
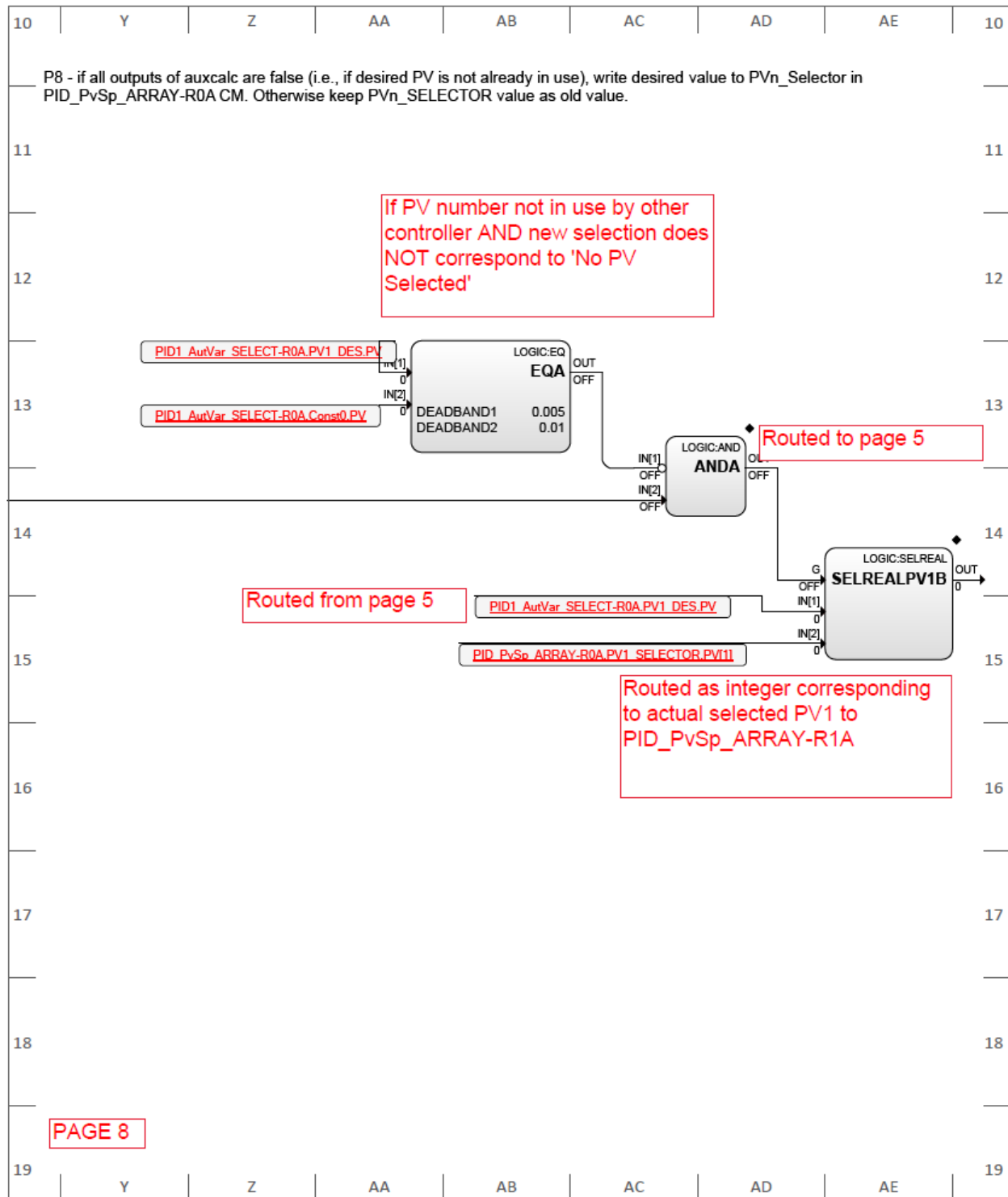


Figure 17: PIDn_AutVar_SELECT-R0A, Page 6.

Figure 18: PID_n_AutVar_SELECT-R0A, Page 7.

Figure 19: PID_n_AutVar_SELECT-R0A, Page 8.

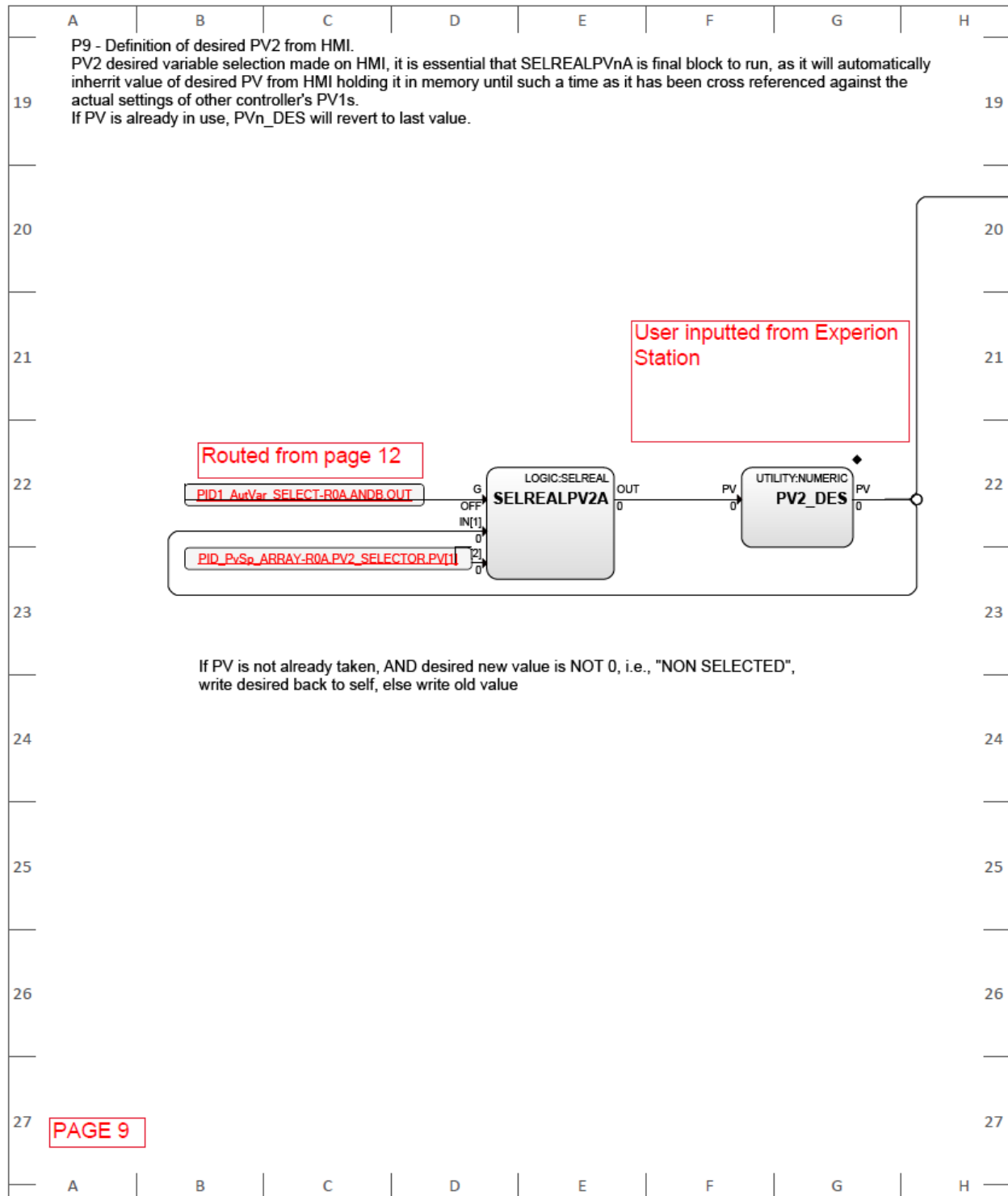


Figure 20: PIDn_AutVar_SELECT-R0A, Page 9.

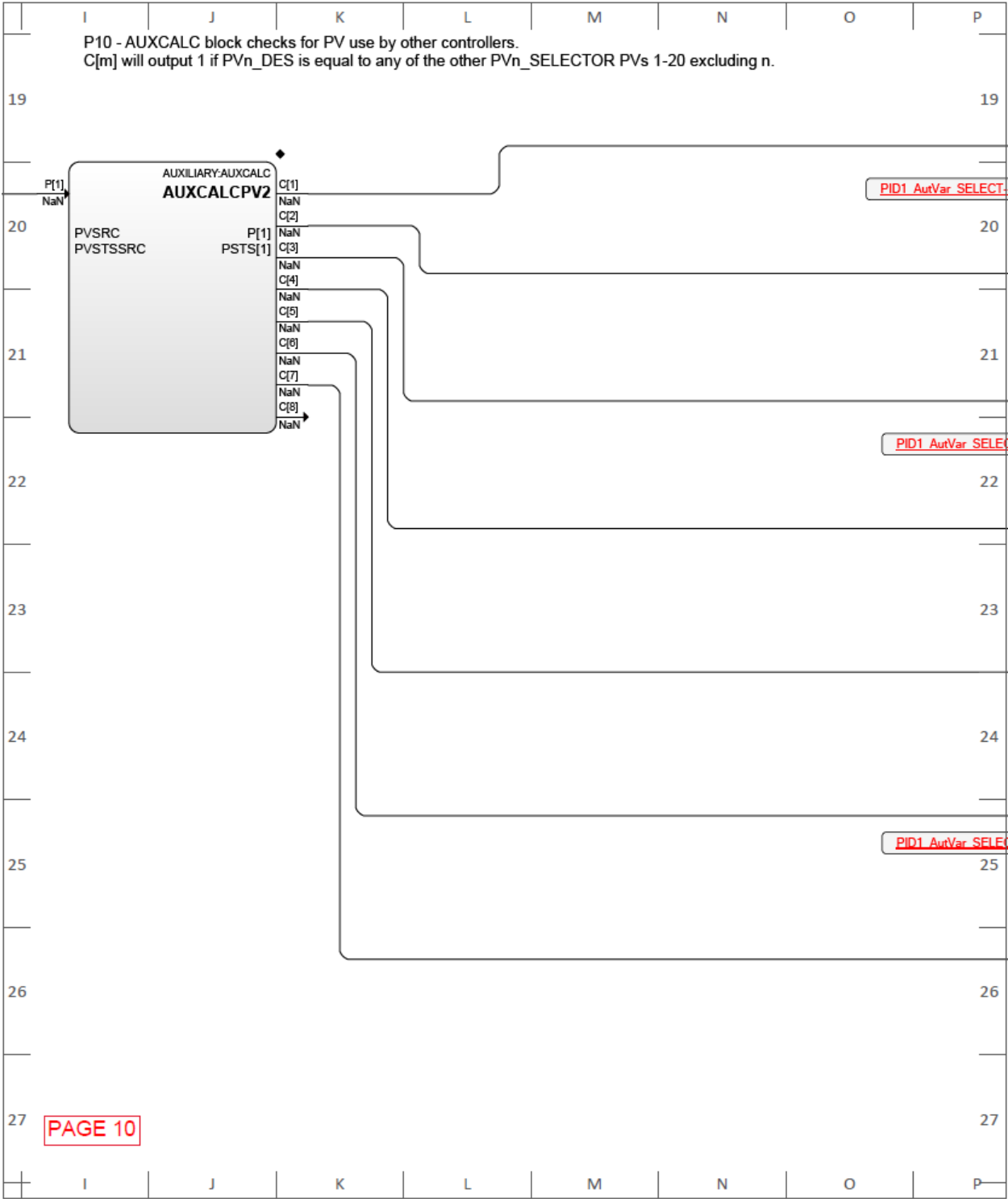
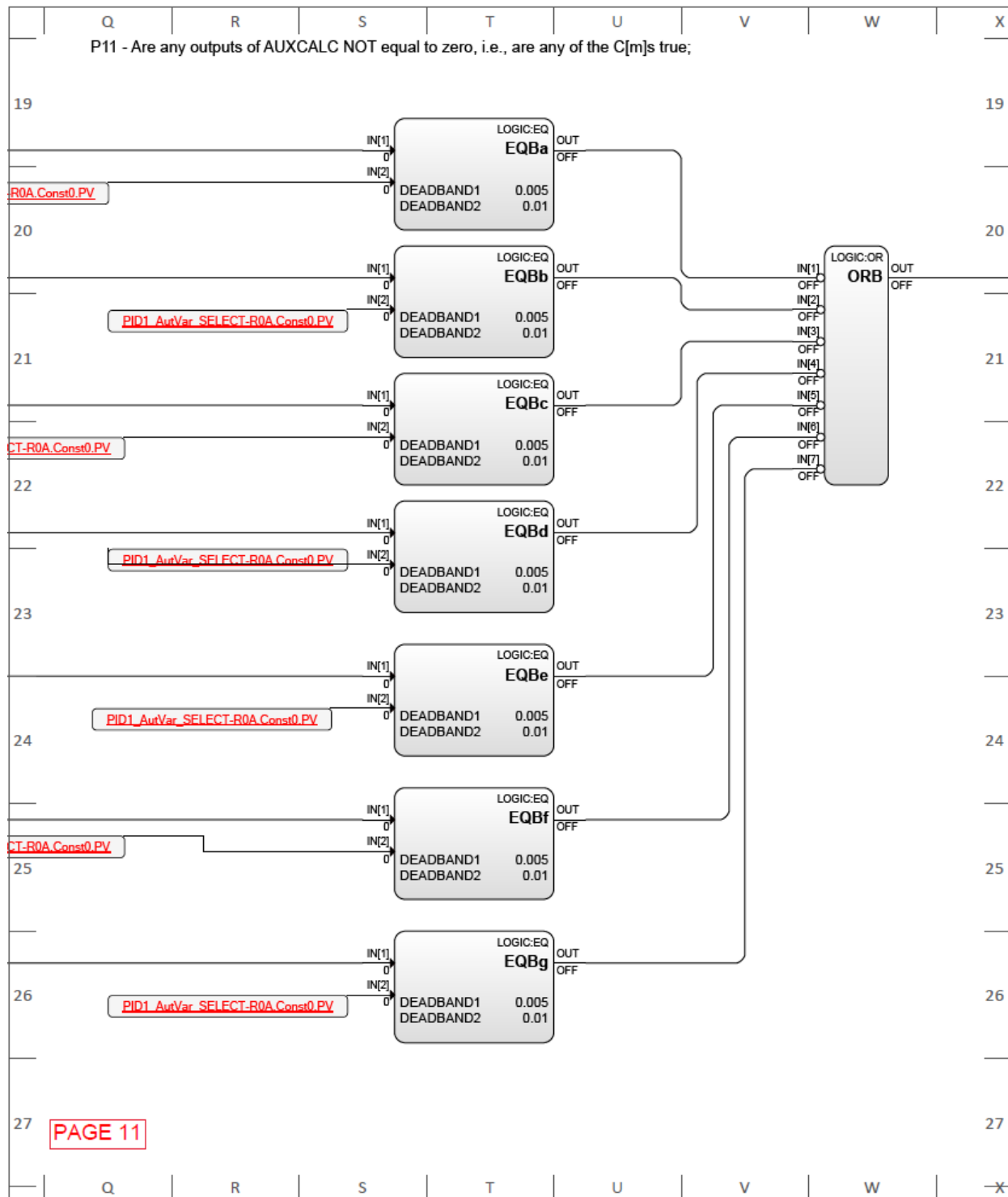


Figure 21: PIDn_AutVar_SELECT-R0A, Page 10.

Figure 22: PID_n_AutVar_SELECT-R0A, Page 11.

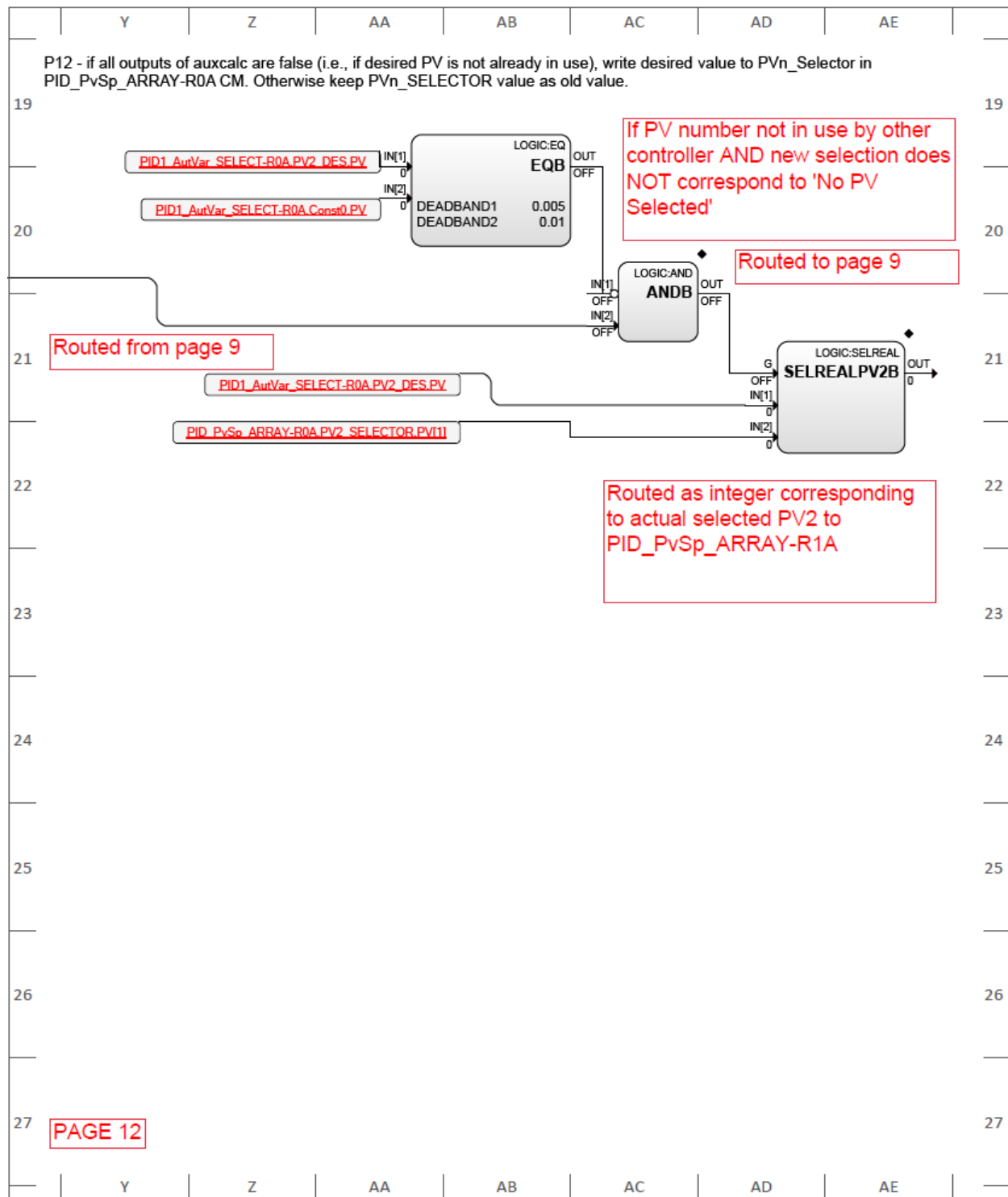


Figure 23: PIDn_AutVar_SELECT-R0A, Page 12.

5.2.3 PID_nPV_SELECT-R1A

For the following explanation, we will track the routing of the integer corresponding to the selected PV1 from its entry into the CM (*Figure 28, Figure 32, Figure 36, Figure 40*) through its transformation into the actual value corresponding to that integer value, and its eventual routing out of the CM (*Figure 25*). The same process is carried out on the integer values corresponding to the selected SP, PV2, and DV, although this will not be covered.

Starting with *Figure 28*, the integer corresponding to the selected PV1 is checked to see if it lies within the range 1-8; similarly, in *Figure 32, Figure 36* and *Figure 40*, the integer corresponding to the selected PV1 is checked to see if it lies within the range 9-16, 17-24, and 25-32, respectively; the purpose of which will be described later.

In *Figure 29*, the integer corresponding to the selected PV1 has the number one subtracted from it to pull it within the range 0-7 (*the 'MUXREAL' block requires an input in the range 0-7 to function, as a result in Figure 33, Figure 37, and Figure 41 subtractions are applied to pull the input to the same range*). Returning to *Figure 29*, the integer ranging from 0-7 is converted to a 32-bit float and fed into a 'MUXREAL' function block. 'MUXREAL' blocks take an integer input ranging from 0-7 and route one of eight possible floating-point inputs to the output. The outputs of all 'MUXREAL' blocks for the PV1 variable (*Figure 29, Figure 33, Figure 37, Figure 41*) are routed to the 'SWITCHPV1' function block (*Figure 25*).

'SWITCH' function blocks work by taking a number of numerical inputs (1, ..., N) and a number of FLAG inputs (1, ..., N) as number/FLAG pairs; the 'SWITCH' function block will then look through the FLAG inputs in ascending order and output the number corresponding to the first FLAG that is in the TRUE state. Thus, the 'SWITCHPV1' block will route the output of the 'MUXREAL' block corresponding to the correct selected PV1 range.

A	B	C	D	E	F	G	H
1	PID1_PV_SELECT-R1A Control Module Description: The purpose of this CM lays in the routing of selected PV, SP, and DV values from MUPP Sensors to the input channels associated with the controller to which the variable selection has been allocated. This routing starts by checking if the PV and DV selection variables are within a specific range (1-8, 9-16, 17-25 or 26-32) a boolean associated with one of the ranges is activated. The range number also informs a MUXREAL block which actual value input to route to the final output switch. The final output switch will output the real value associated with the selection boolean which is active.						1
2							2
3	KEY FUNCTIONS: TITLE PAGE PV1 & SP Output Routing to PID_PvSp_ARRAY CM PV2 Output Routing to PID_PvSp_ARRAY CM DV Output Routing to PID_PvSp_ARRAY CM Check For PV, DV Selection Range 1<x<8 Route Actual PV1/SP for Selection 1<x<8 to P2 Route Actual PV2 for Selection 1<x<8 to P3 Route Actual DV for Selection 1<x<8 to P4 Check For PV, DV Selection Range 9<x<16 Route Actual PV1/SP for Selection 9<x<16 to P2 Route Actual PV2 for Selection 9<x<16 to P3 Route Actual DV for Selection 9<x<16 to P4 Check For PV, DV Selection Range 17<x<24 Route Actual PV1/SP for Selection 17<x<24 to P2 Route Actual PV2 for Selection 17<x<24 to P3 Route Actual DV for Selection 17<x<24 to P4 Check For PV, DV Selection Range 25<x<32 Route Actual PV1/SP for Selection 25<x<32 to P2 Route Actual PV2 for Selection 25<x<32 to P3 Route Actual DV for Selection 25<x<32 to P4						3
4	PAGE 1 PAGE 2 PAGE 3 PAGE 4 PAGE 5 PAGE 6 PAGE 7 PAGE 8 PAGE 9 PAGE 10 PAGE 11 PAGE 12 PAGE 13 PAGE 14 PAGE 15 PAGE 16 PAGE 17 PAGE 18 PAGE 19 PAGE 20						4
5							5
6							6
7	REFERENCES: AUTHOR: NICHOLAS O'KEEFFE <i>As part of thesis WU3, 2022</i> REVISION HISTORY: Made with some reference to PID1_PV_SELECT (R0) <i>Written for the PLC5 to C300 migration.</i>						7
8							8
9	PIDn_PV_SELECT-R1A exists for each of the twenty configured controllers, only variable names change.						9
A	B	C	D	E	F	G	H

Figure 24: PIDn_PV_SELECT-R1A, Page 1.

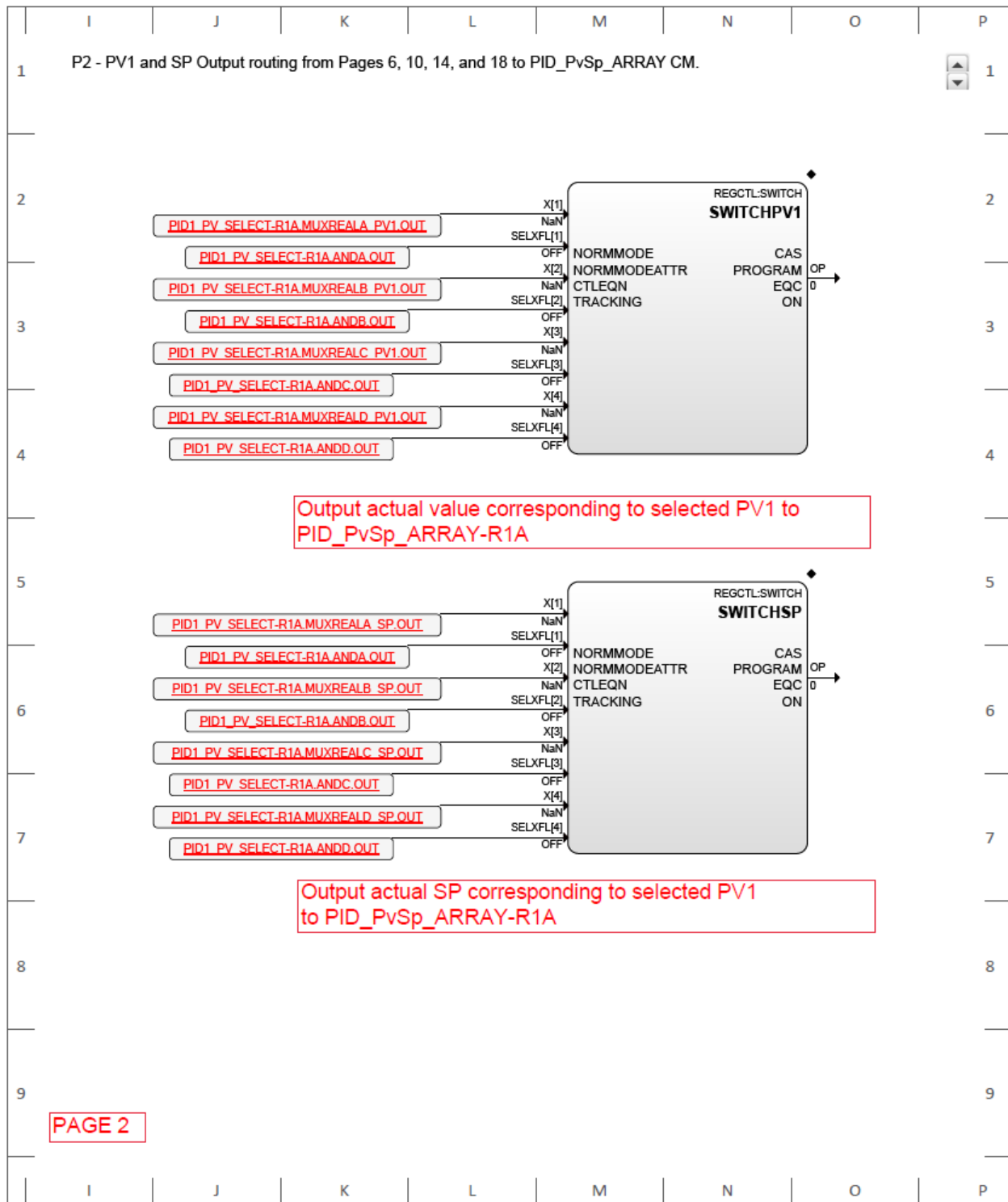


Figure 25: PIDn_PV_SELECT-R1A, Page 2.

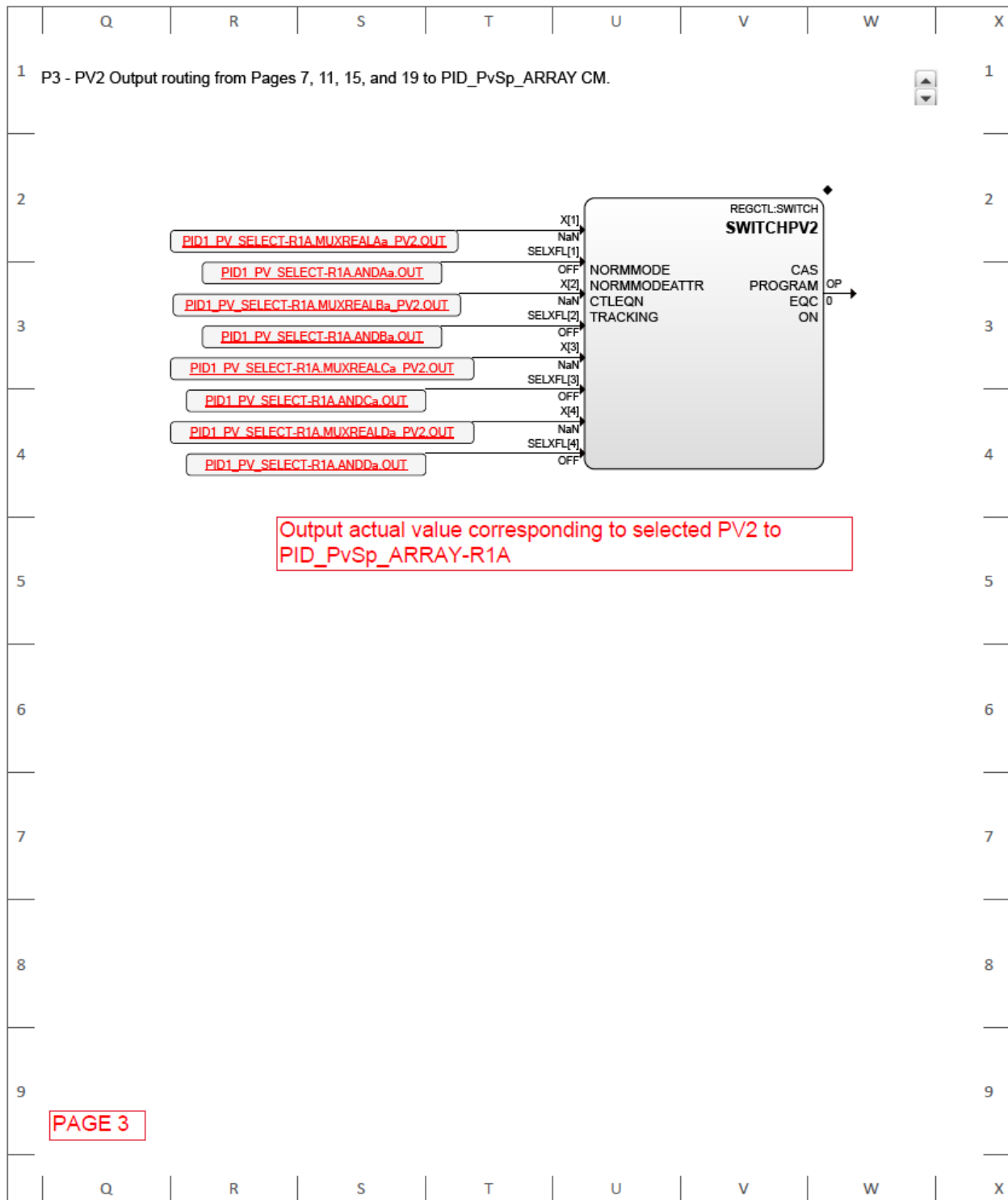


Figure 26: PIDn_PV_SELECT-R1A, Page 3.

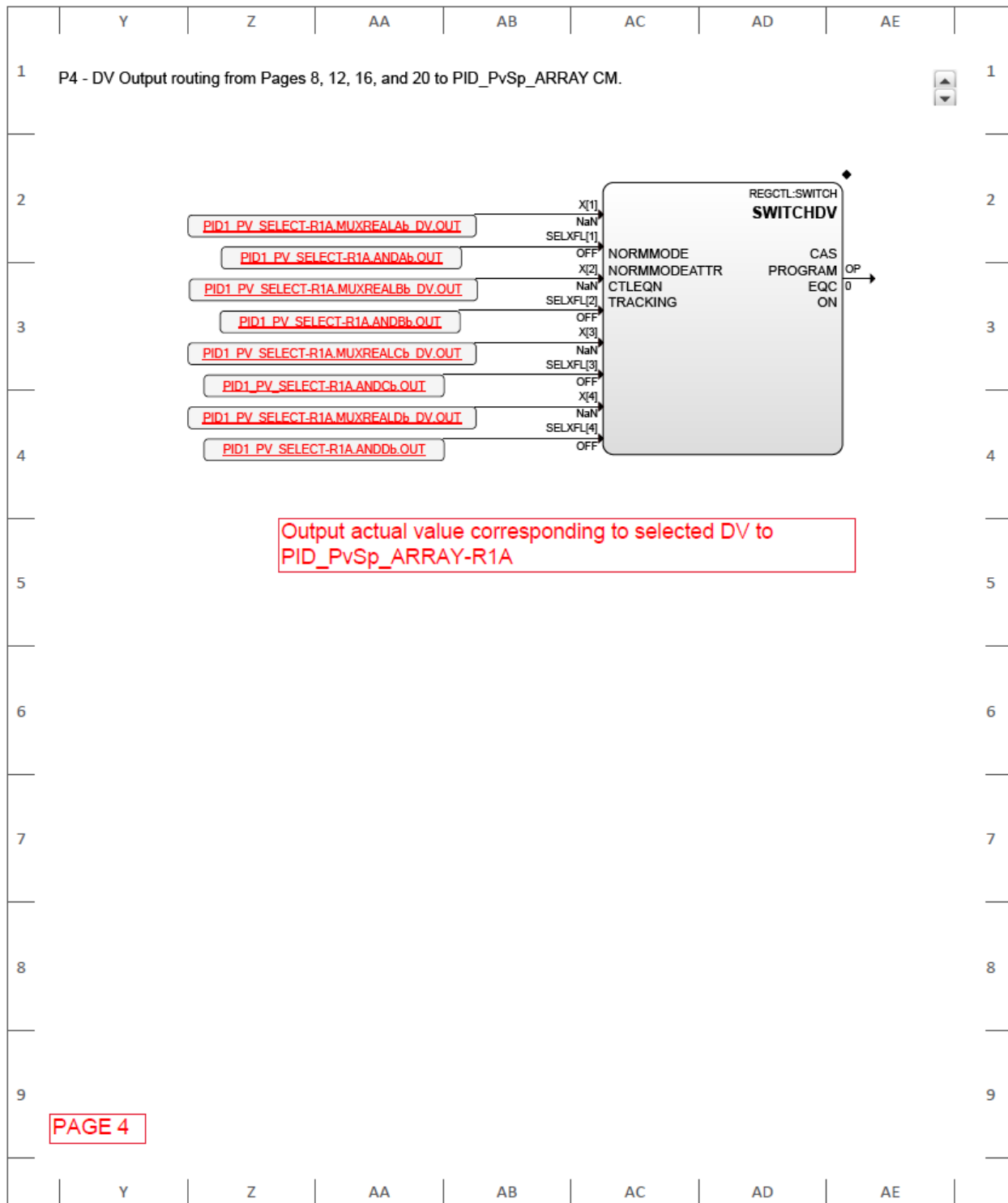


Figure 27: PIDn_PV_SELECT-R1A, Page 4.

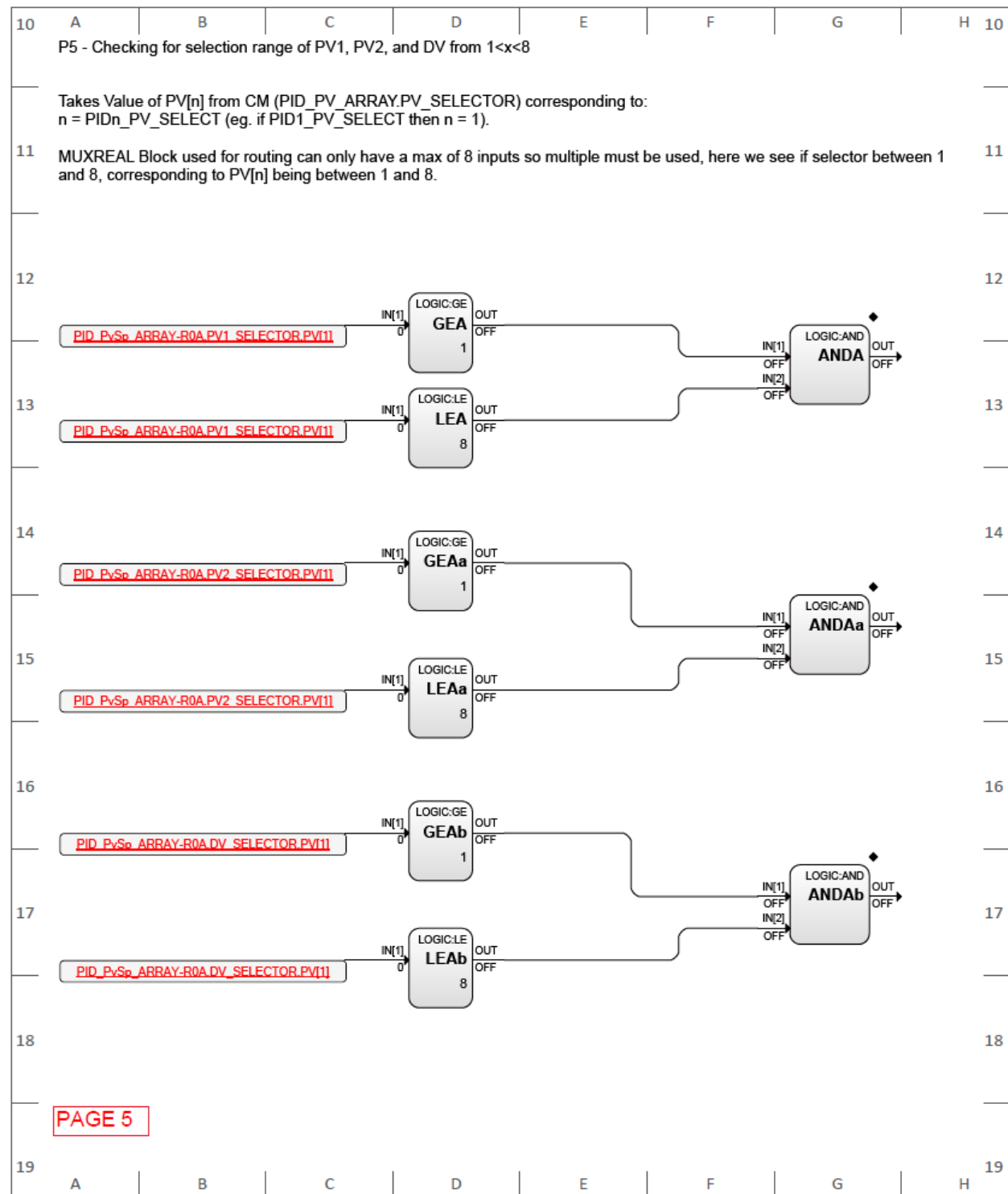


Figure 28: PIDn_PV_SELECT-R1A, Page 5.

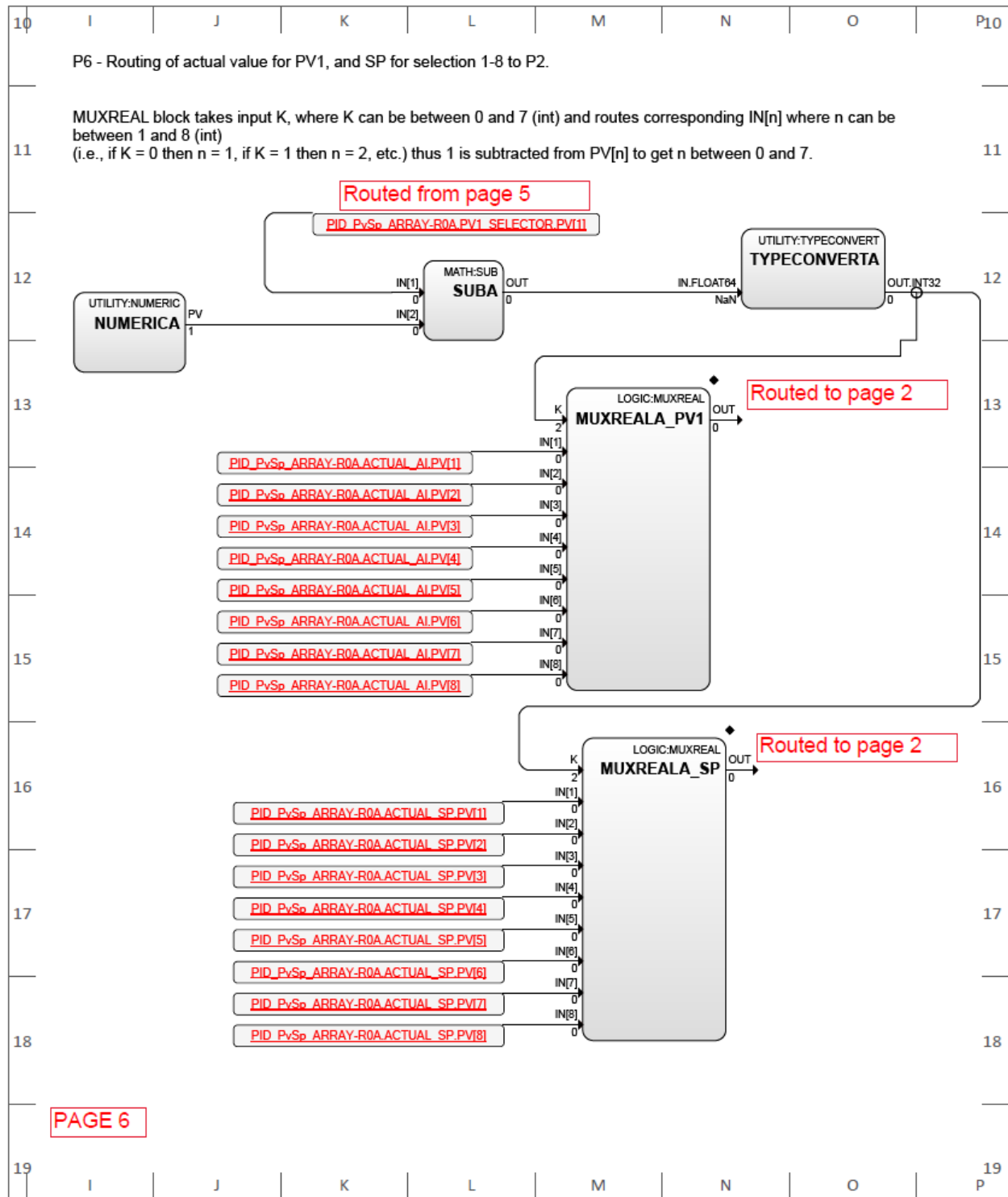


Figure 29: PIDn_PV_SELECT-R1A, Page 6.

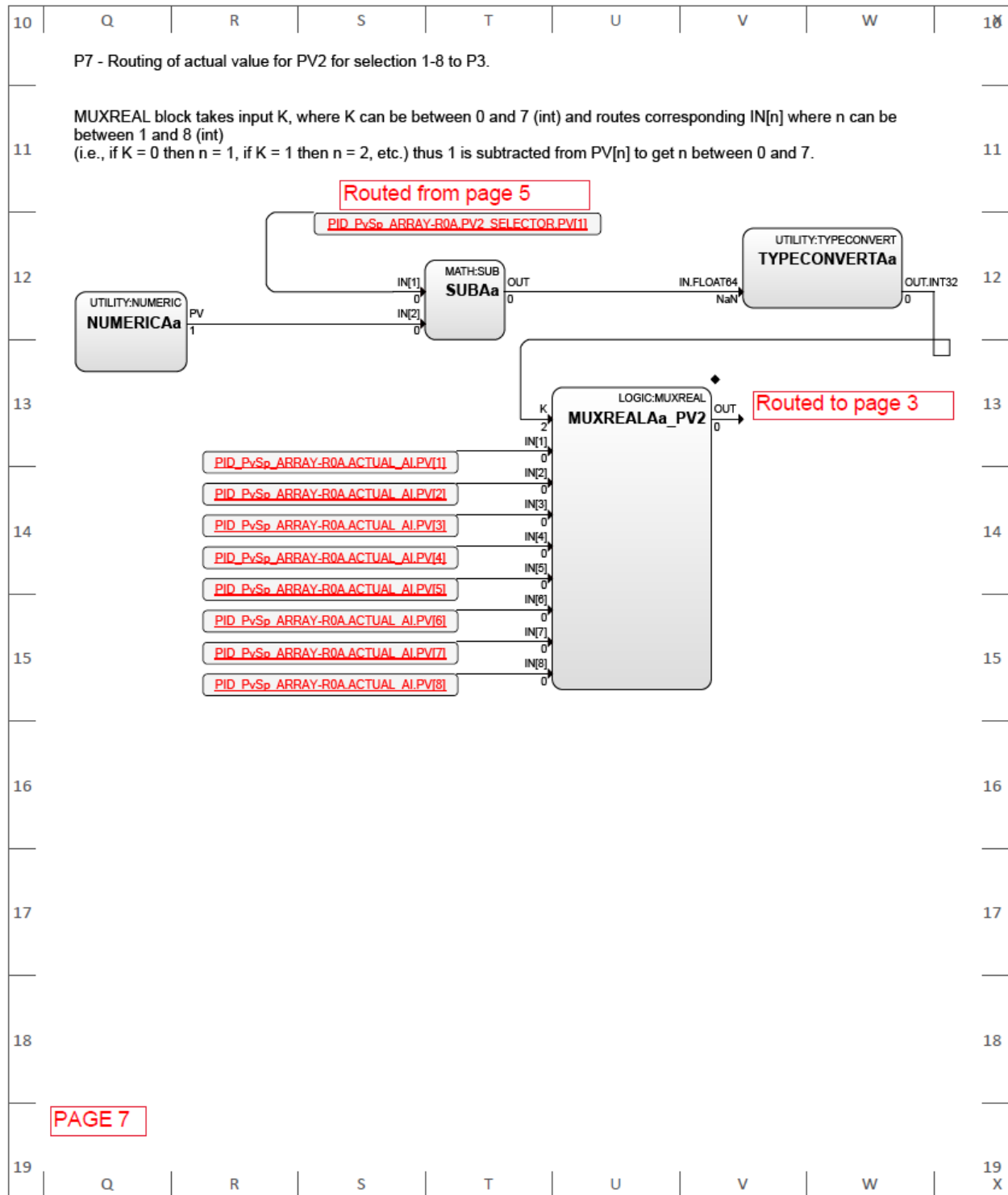


Figure 30: PIDn_PV_SELECT-R1A, Page 7.

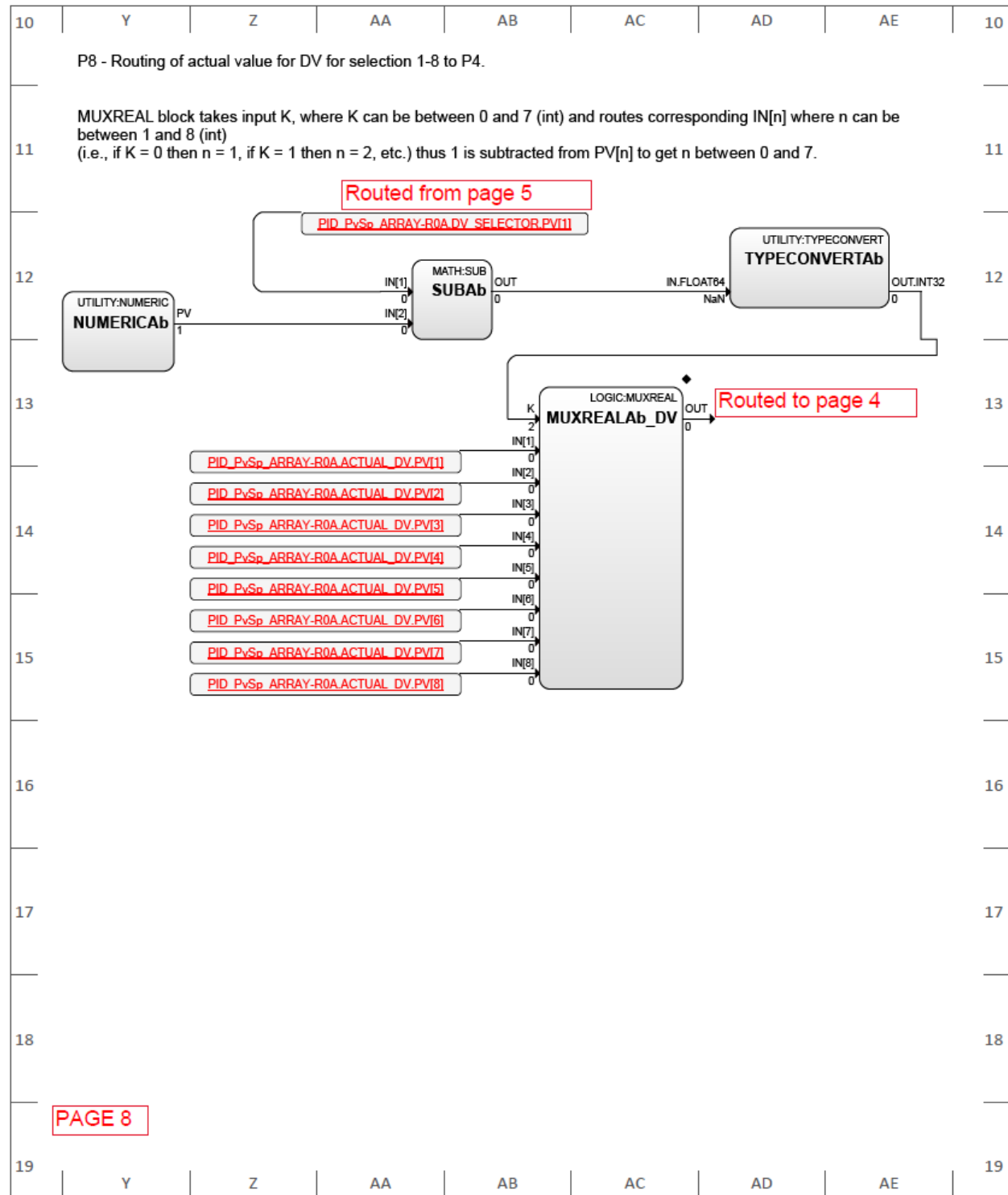


Figure 31: PIDn_PV_SELECT-R1A, Page 8.

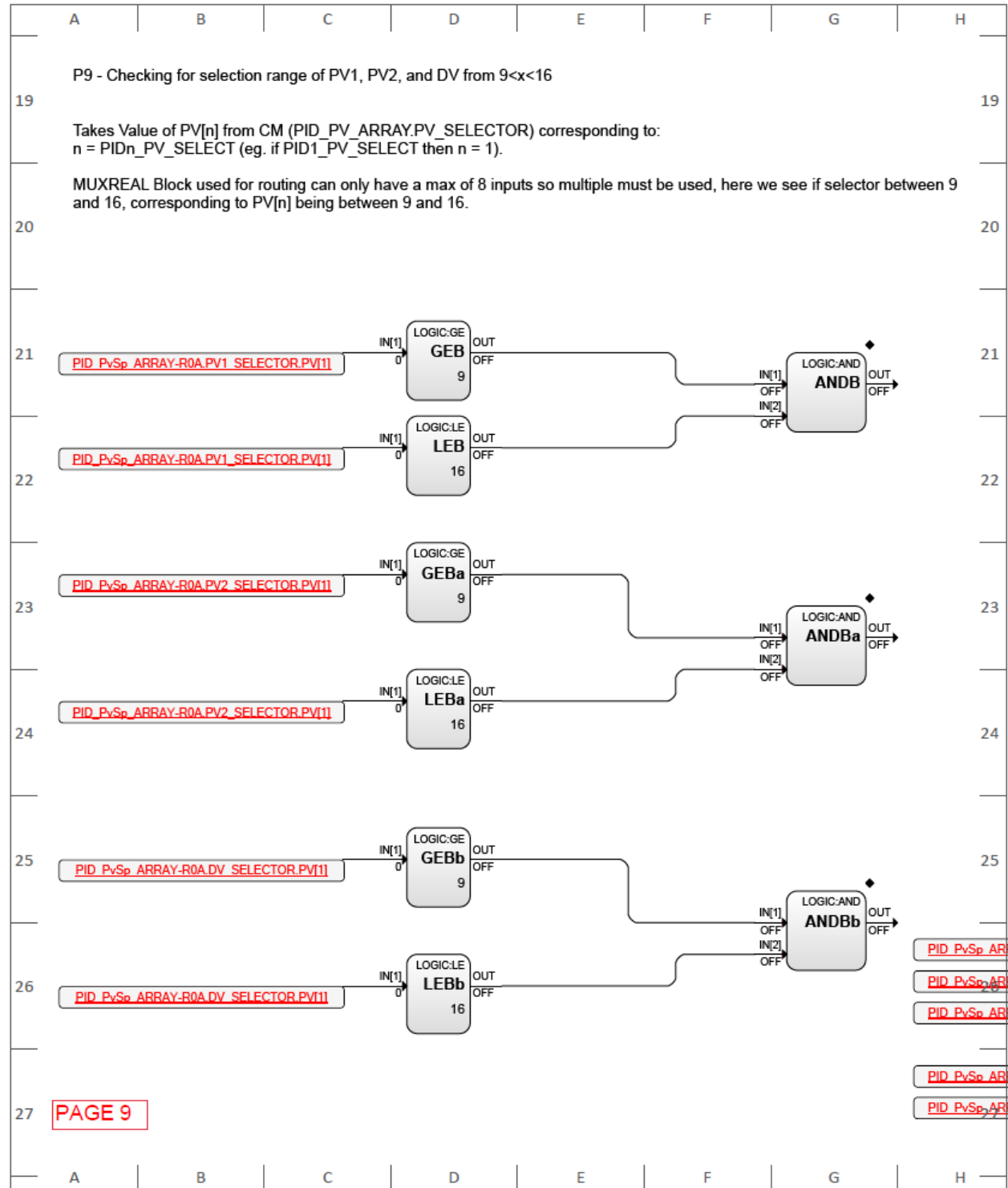


Figure 32: PIDn_PV_SELECT-R1A, Page 9.

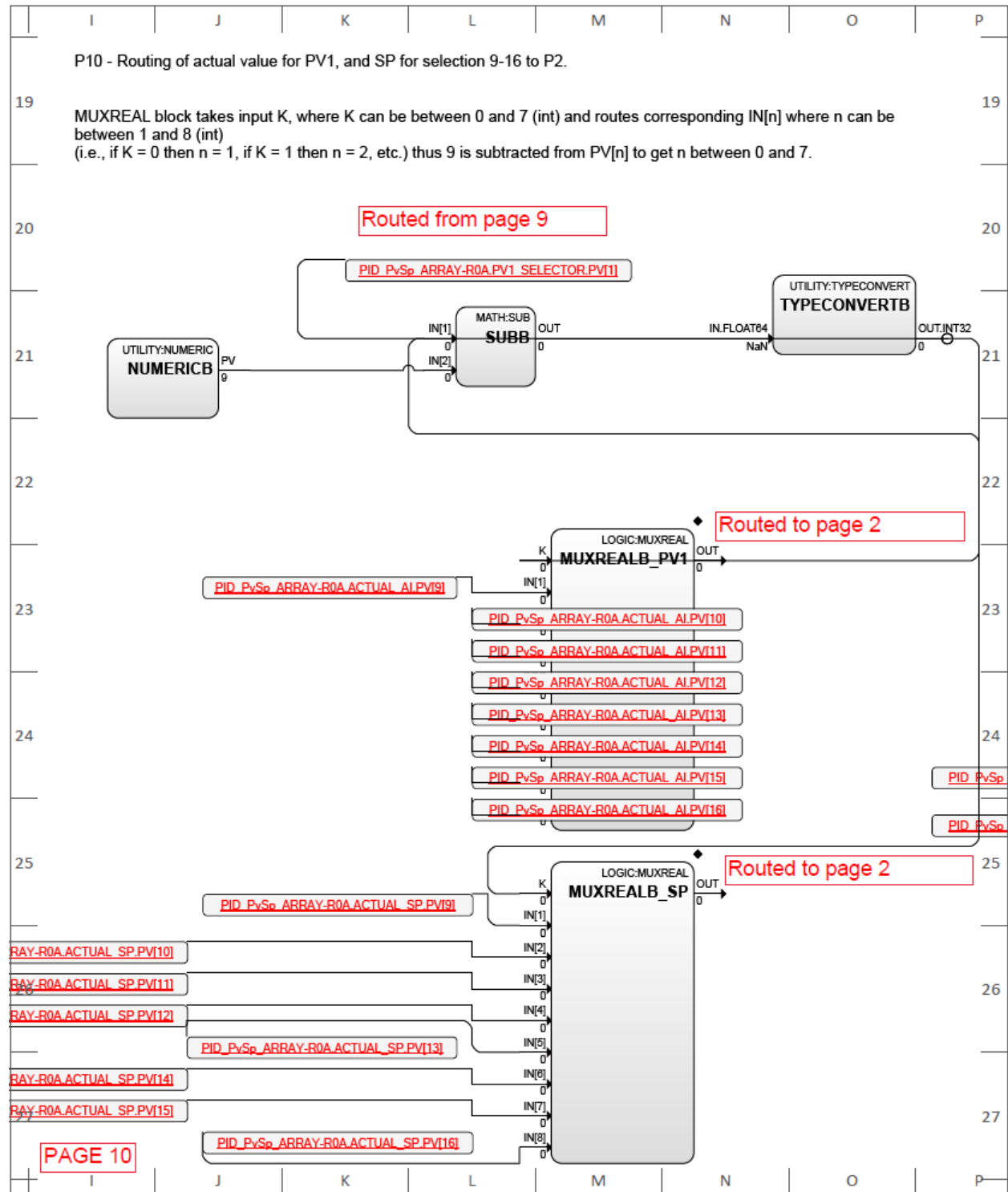


Figure 33: PIDn_PV_SELECT-R1A, Page 10.

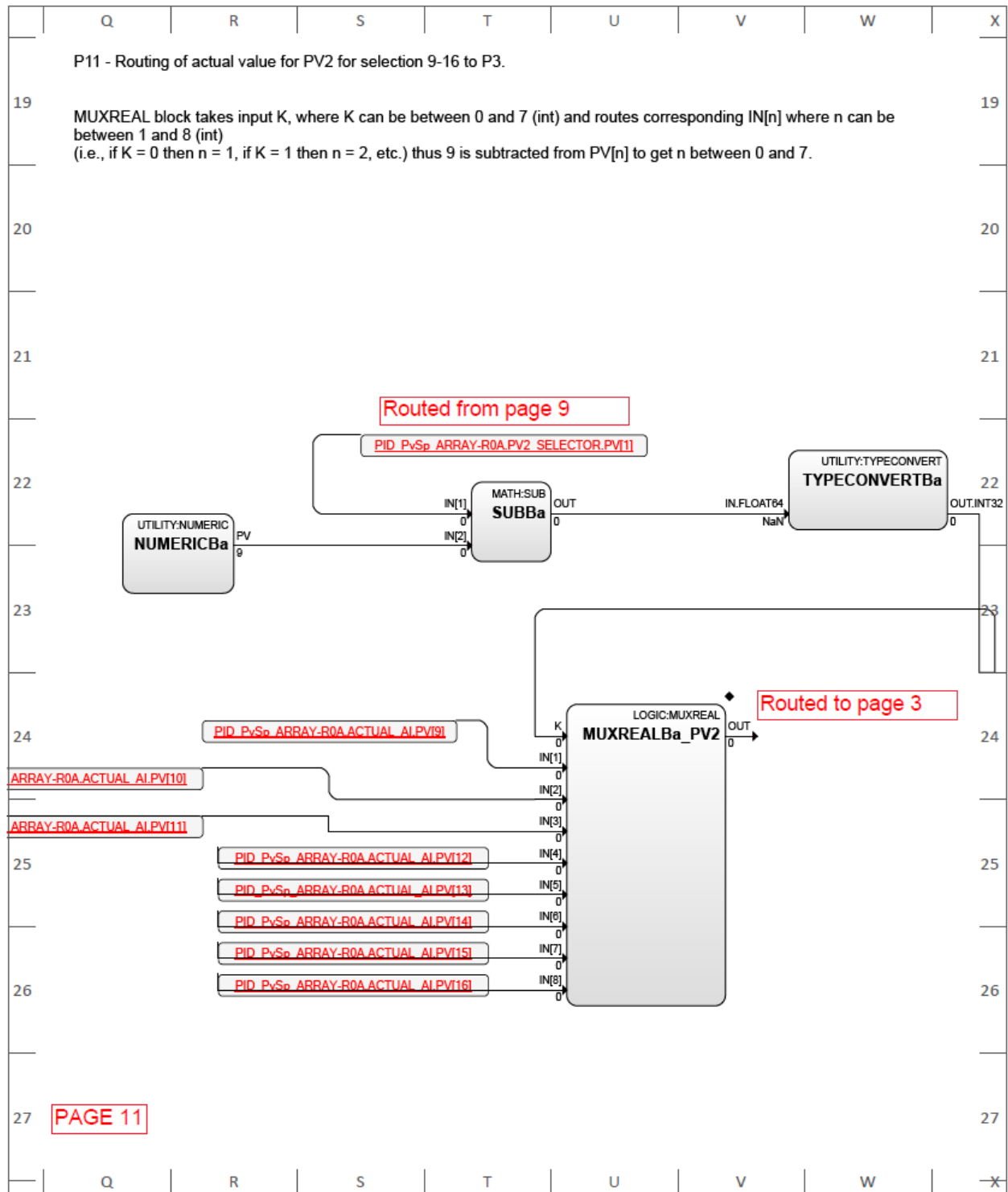


Figure 34: PIDn_PV_SELECT-R1A, Page 11.

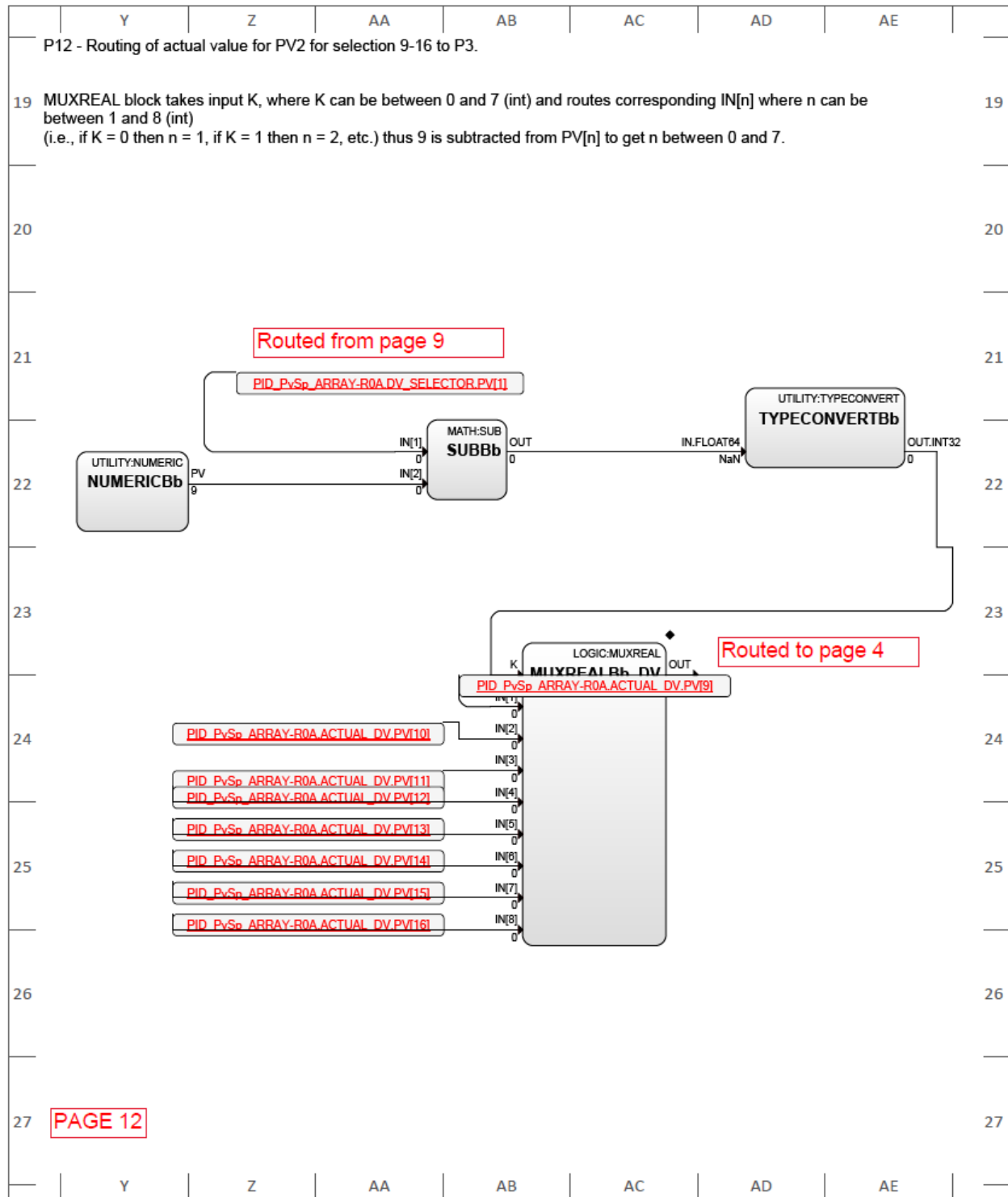


Figure 35: PIDn_PV_SELECT-R1A, Page 12.

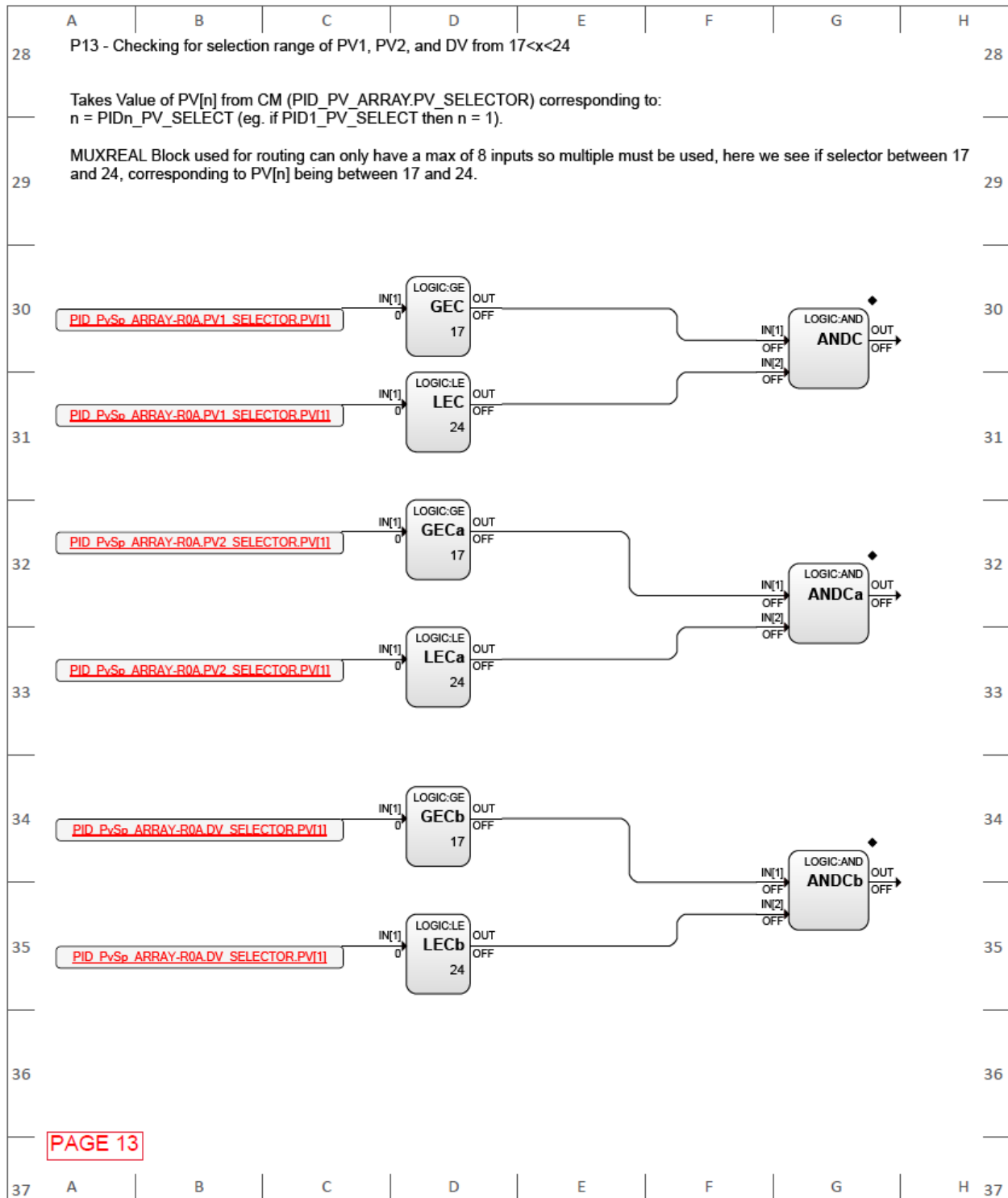


Figure 36: PIDn_PV_SELECT-R1A, Page 13.

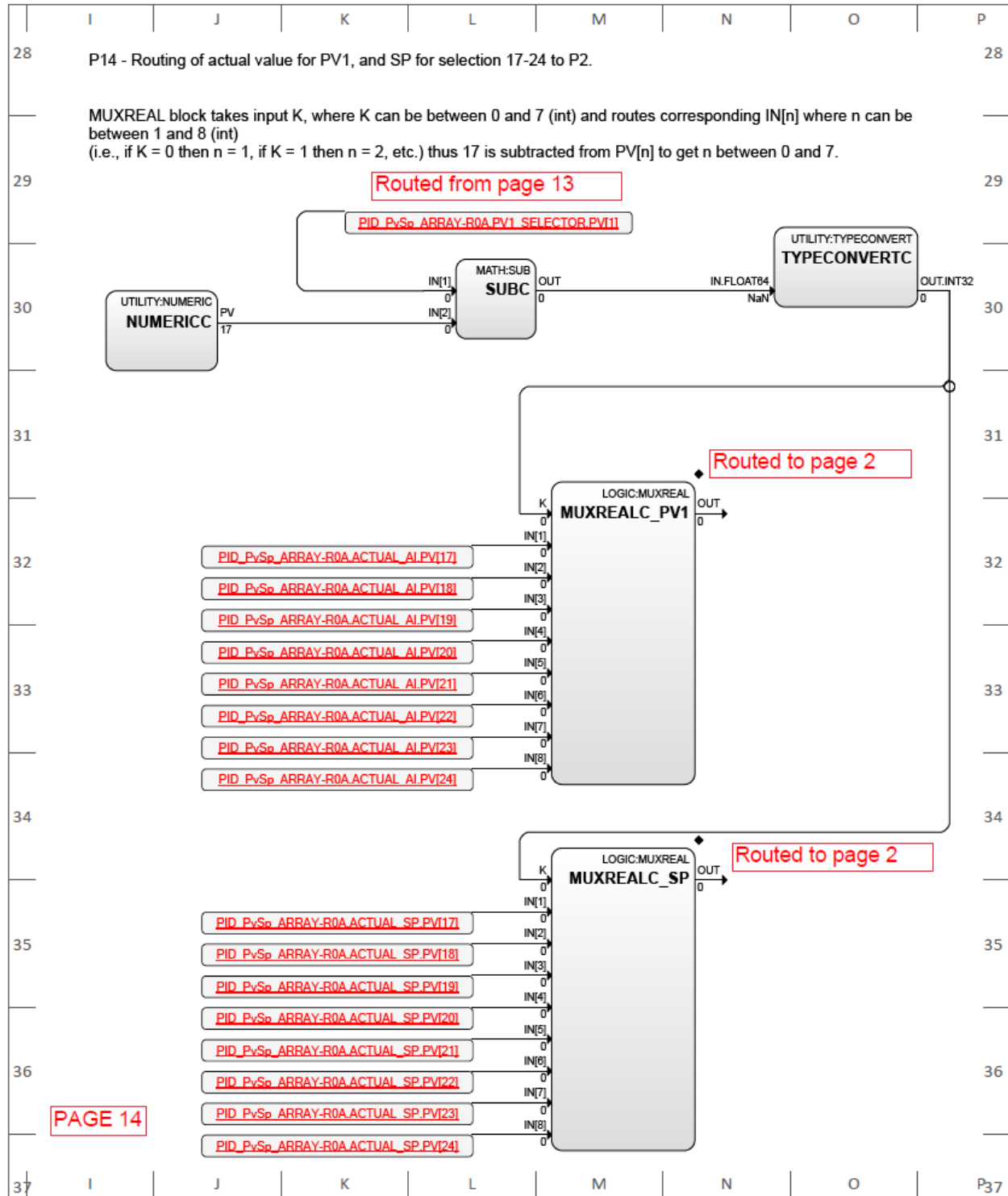


Figure 37: PIDn_PV_SELECT-R1A, Page 14.

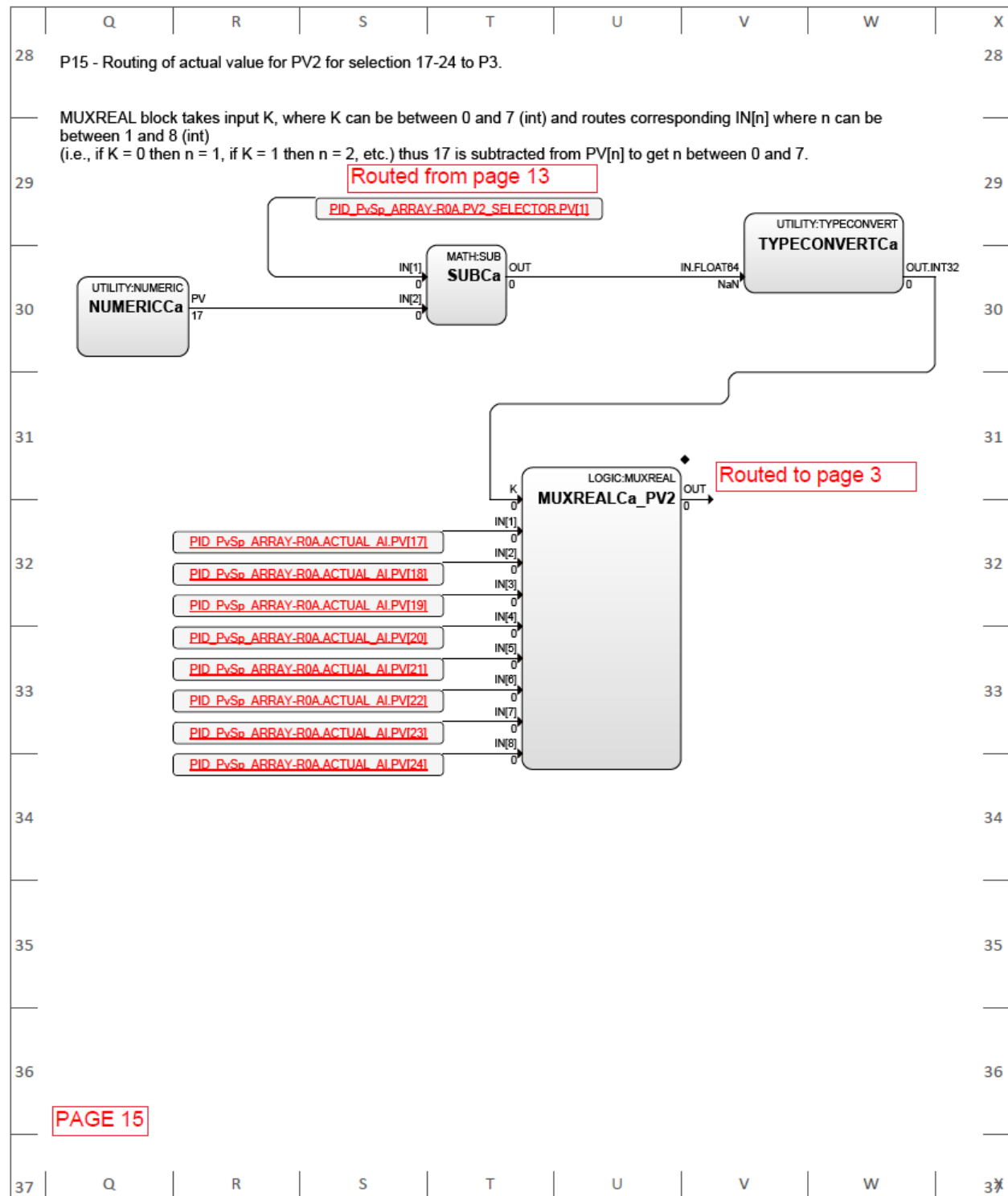


Figure 38: PIDn_PV_SELECT-R1A, Page 15.

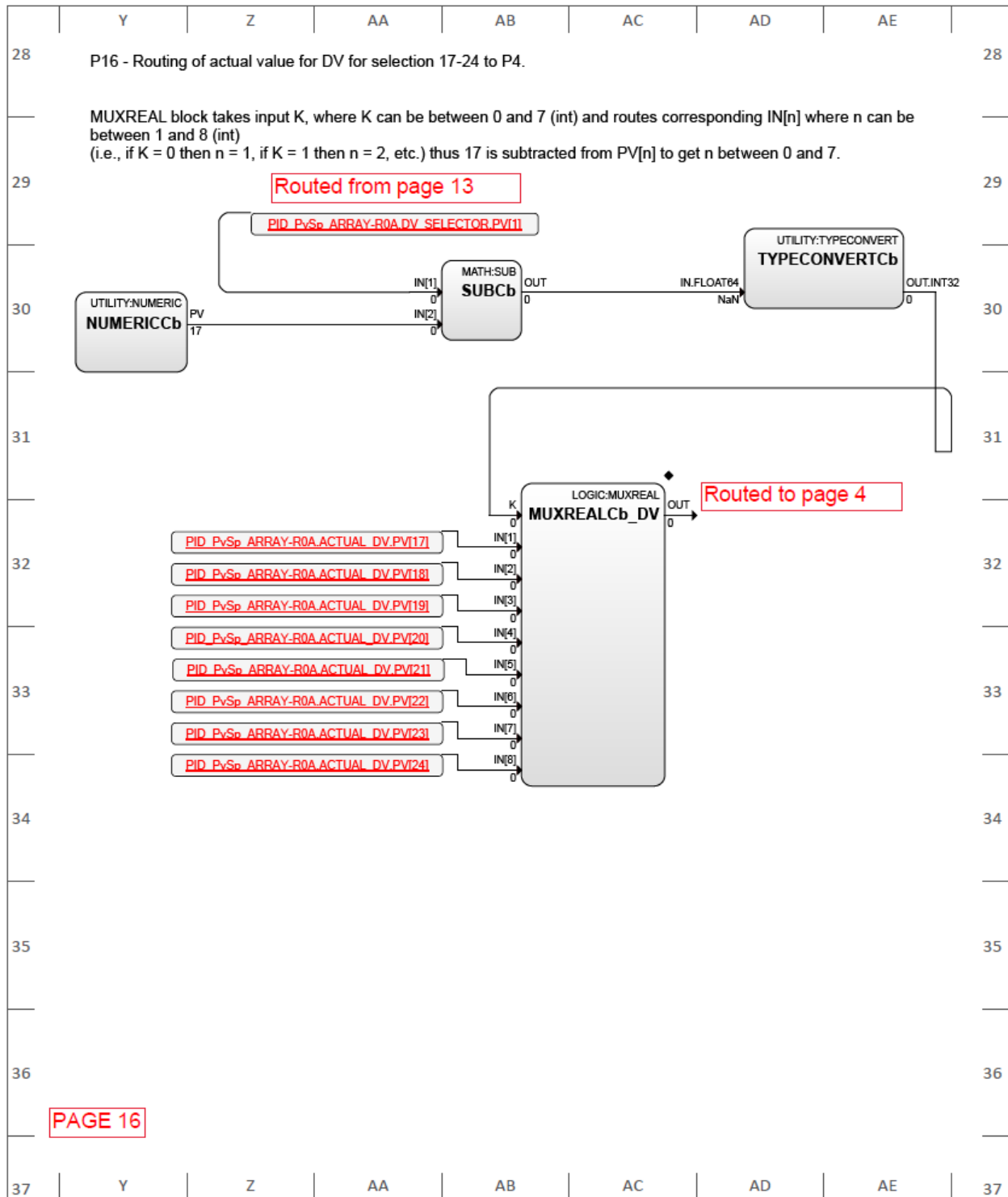


Figure 39: PIDn_PV_SELECT-R1A, Page 16.

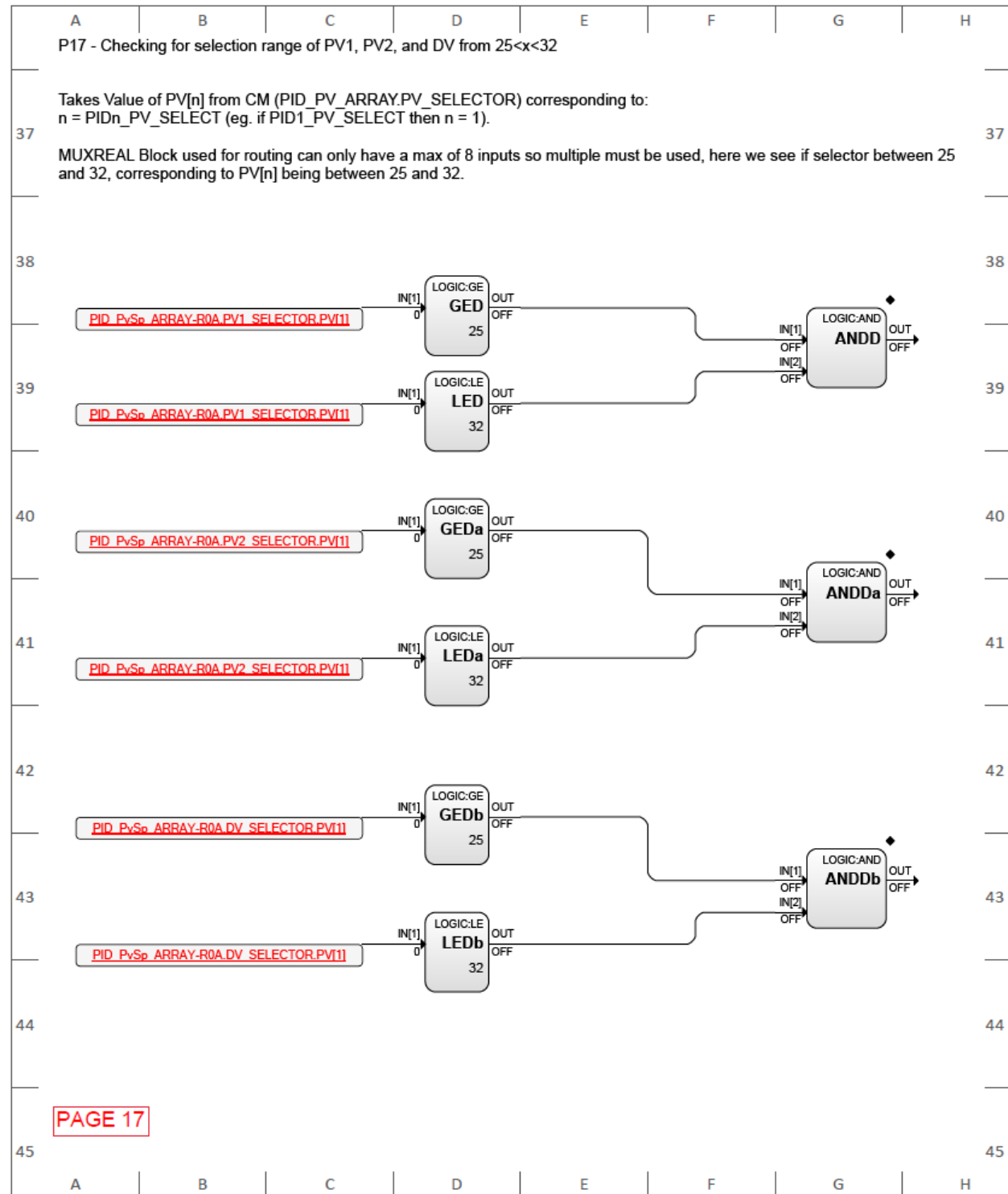


Figure 40: PIDn_PV_SELECT-R1A, Page 17.

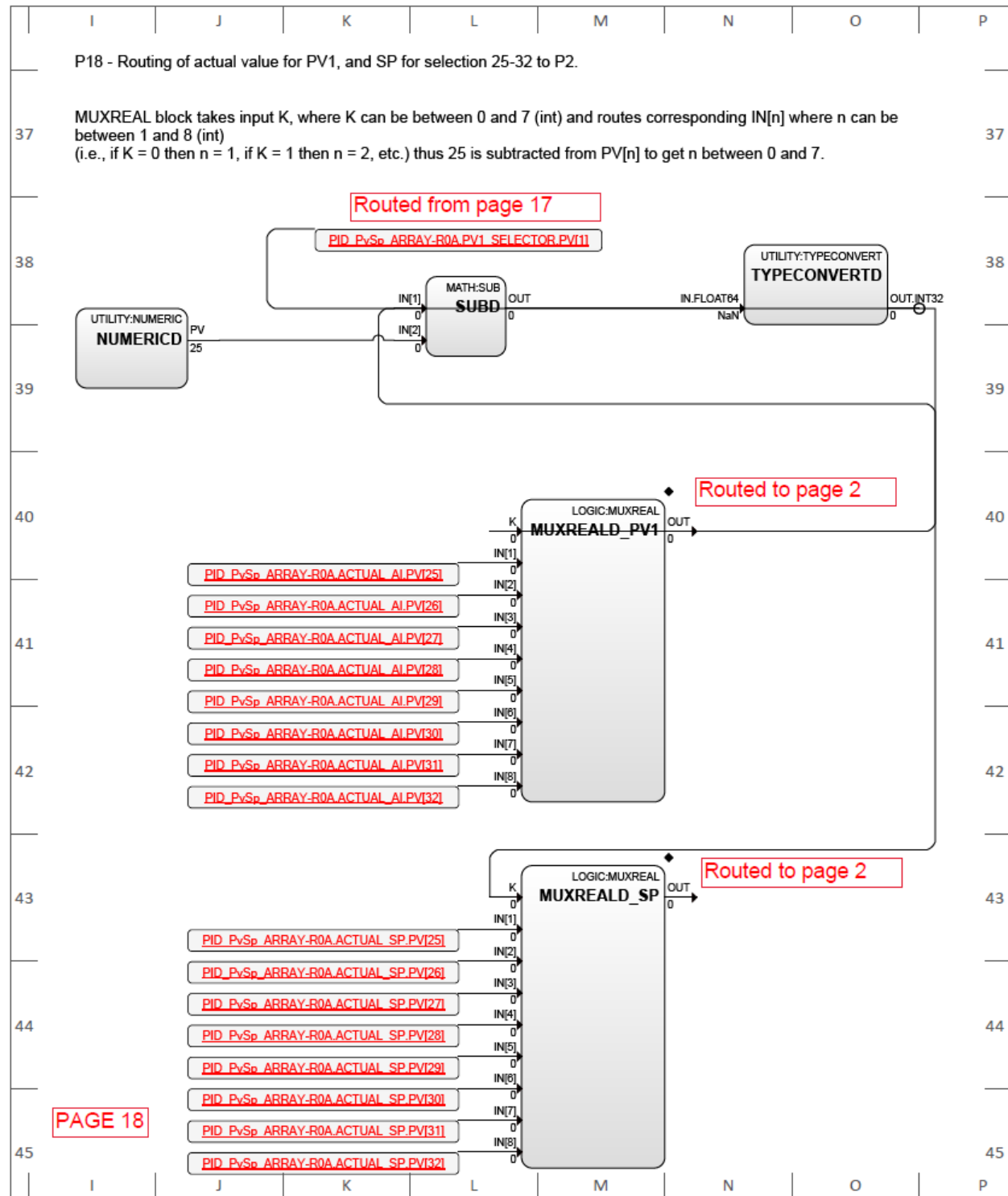


Figure 41: PIDn_PV_SELECT-R1A, Page 18.

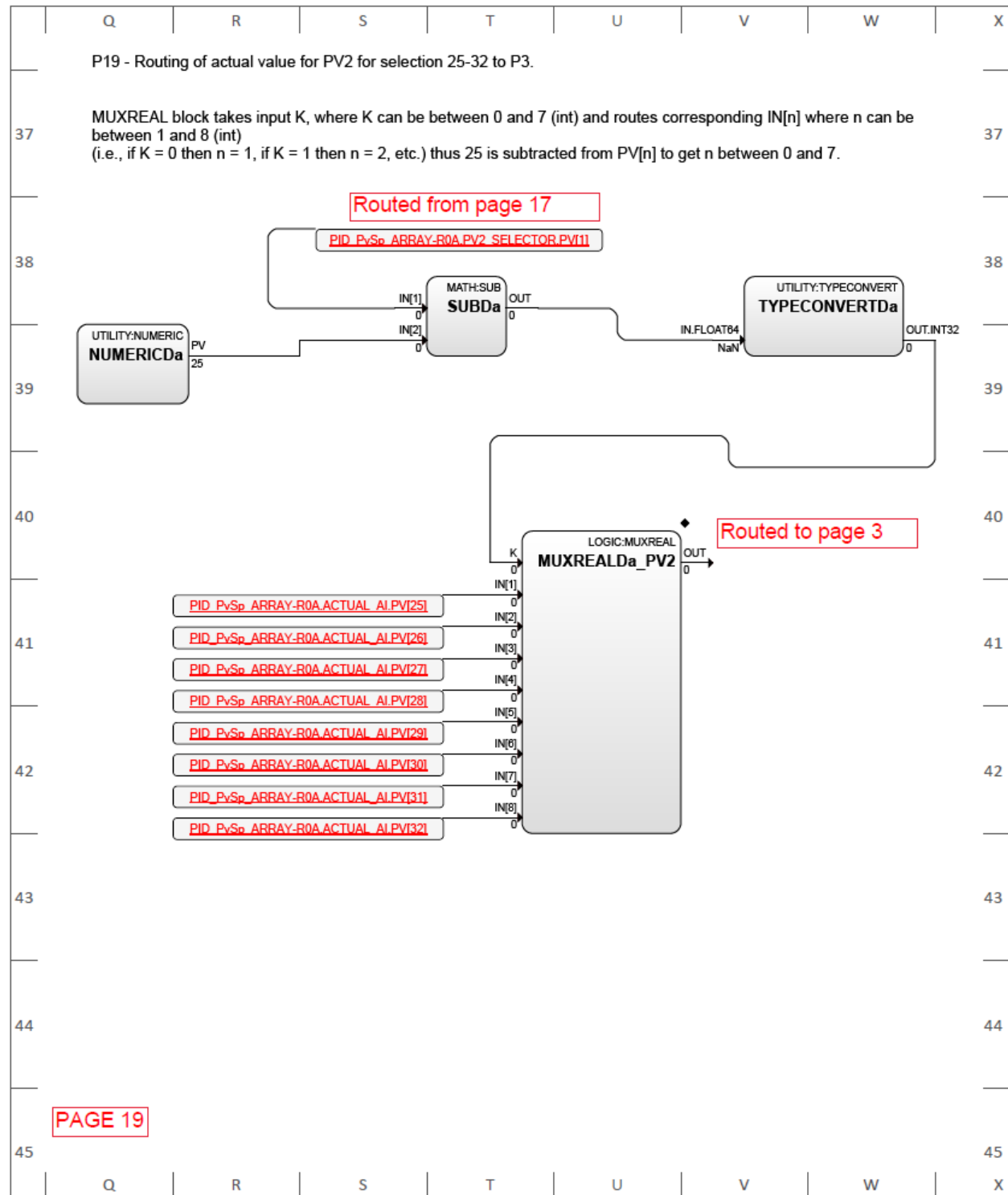


Figure 42: PIDn_PV_SELECT-R1A, Page 19.

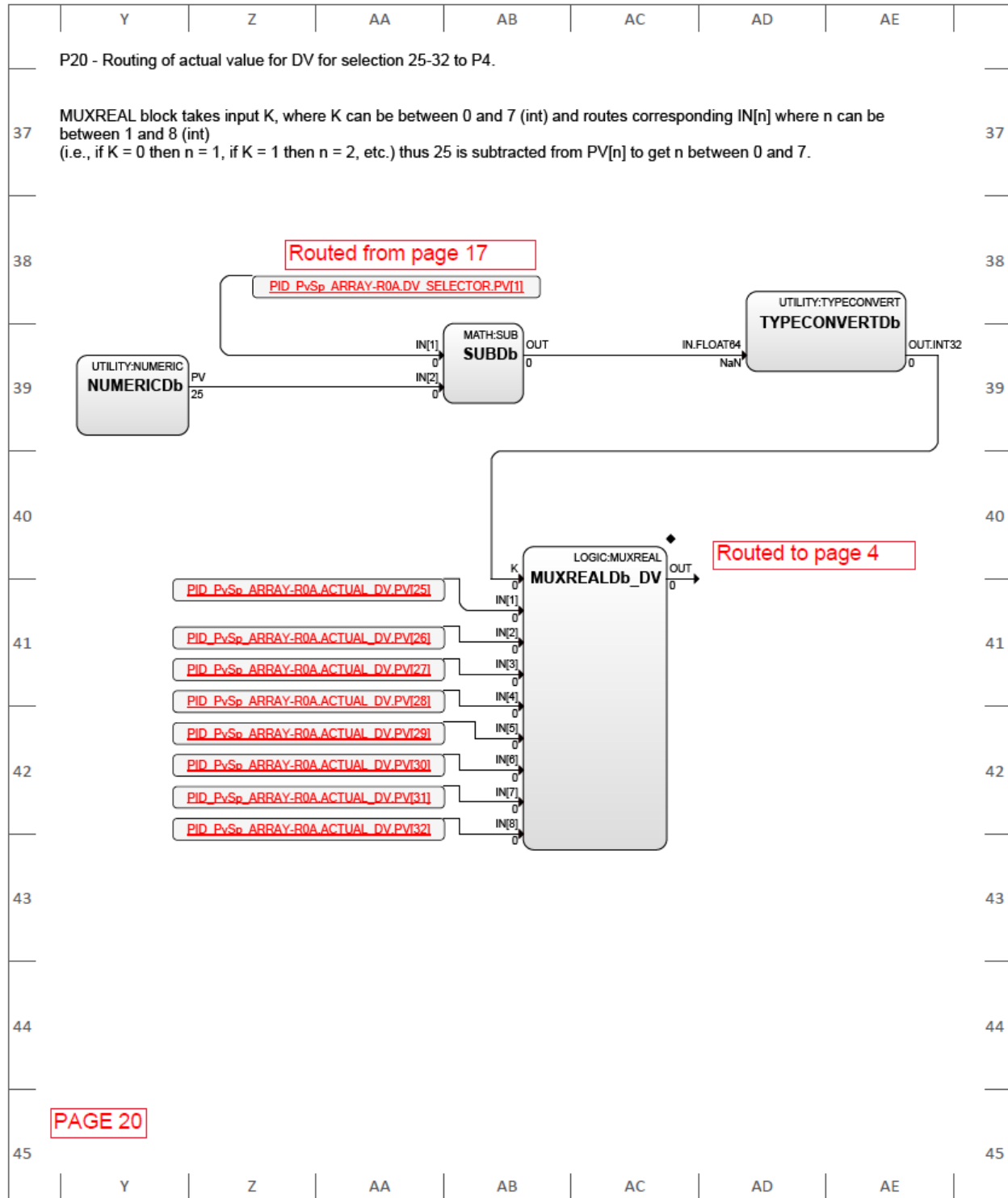


Figure 43: PIDn_PV_SELECT-R1A, Page 20.

5.2.4 Final Control Element CM

The actual number values corresponding to the selected PV1, PV2, and DV are routed from **PID_PvSp_ARRAY-R1A**, and the SP is retrieved from EPKS Station (Figure 49). Observing Figure 50, we can see the DV value is passed through a 'LEADLAG' and 'DEADTIME' function block, which, when combined, create a (2, 1) plus delay transfer function; the other inputs to these blocks constitute the lead time, and lag time constants, and delay, and are retrieved from EPKS Station. The value of the 'DEADTIME' block is outputted into the second input of a 'SELREAL' block, which outputs this value IF feedforward control is enabled in EPKS Station; otherwise, the 'SELREAL' block outputs zero; the block is fed into the 'PIDFF' block as the feedforward component of the block (Figure 46).

The 'PIDFF' function block (Figure 46) receives its control parameters from Station, SP and PV inputs from the 'SP' and 'PV1' numeric blocks (Figure 49) and is enabled by the 'AUTO' FLAG retrieved from Station (Figure 45). The operating point value from 'PIDFF' is transferred to the first input of a 'SELREAL' block; the other input comes from the 'NUMERIC' block "FCV570_OPC". This OPC block serves the purpose of receiving data over OPC and constitutes the output of the OPC outer-loop controller (Figure 46). The 'SELREAL' block's output depends on the 'OPCenable' FLAG (Figure 45), which is routed to the "MV_OuterLoop" NUMERIC. The "MV_OuterLoop" NUMERIC becomes the setpoint in the inner-loop PID controller "PIDA", which behaves in much the same way as the 'PIDFF' function block, this time being output to the "MV_PIDCAS" NUMERIC (Figure 48).

Turning the focus to Figure 51, we can see where the actual transformation of control topology is taking place. Depending on the FLAG inputs into the 'SWITCH' function block, either the "SP", "MV_OuterLoop", or "MV_PIDCAS" NUMERICS will be routed toward the CM output; these corresponding to manual, single-loop FB or FBFF, or cascade-loop FB or FBFF control schemes, respectively. The 'SWITCH' block moves through one or several 'SELREAL' blocks (*dependent on the control element in question*), which can route NUMERICS from other pre-2022 CMs to keep backward compatibility with previously configured automatic modes in other CMs (Figure 52).

The final step of the CM is dependent on whether the control element is an FCV or pump:

1. For FCVs (Figure 53), the output is routed through interlocks (*implemented with 'SELREAL' blocks*) before moving through a 'NUMERIC' block which acts as a container, storing the value for historical archiving or OPC control purposes. After moving through the 'NUMERIC' block, the CM output moves into the 'SERIES_C_IO:AOCHANNEL' block, which sends the signal to the controller's HART analog output.
2. For pumps (Figure 54), the output is routed directly through a 'NUMERIC' block which acts as a container, before getting scaled from between 0-105 to 0-17199; this number is then fed into the 'PCDI:PCDINUMARRCH' which sends the signal to the controllers MODBUS/TCP output.

	A	B	C	D	E	F	G	H
1	FCV_570-R1A							1
	Control Module Description:							
	This control module serves the purpose of housing the controllers for the flow control valve between the needle tank underflow pump and CSTR-1. It also serves the purpose of routing variables to said FCV position dependent on the parameters configured inside this and other CMs.							2
	KEYFUNCTIONS:							
	TITLE PAGE DOCUMENTATION				PAGE 1			
3	Controller Vs Manual Control Routing				PAGE 2			
	Main Loop PID + FF Controller				PAGE 3			
	Cascade Vs Single Loop Control Routing				PAGE 4			
4	Inner Loop PID Controller				PAGE 5			
	Controller Inputs and Constants (SP, PVs, DVs, etc.)				PAGE 6			
	Feedforward Model, (2,1) + Delay and Routing				PAGE 7			
5	Controller Output Selection				PAGE 8			
	Additions for Backwards Compatibility of REV0 Functions				PAGE 9			
	Output to VSD Interfacing IO				PAGE 10			
6	References:							
	AUTHOR:		NICHOLAS O'KEEFFE <i>As part of Thesis WU3, 2022</i>					
	REVISION HISTORY:		Made with some reference to CUFP_REF_361 (R0) Written for the PLC5 to C300 Migration.					
7								
8								
9								9
	A	B	C	D	E	F	G	H

An example of a final control element CM, a total of 17 such elements exist on the C300 corresponding to 8 pumps:

BMP_REF_241-R1A
CRP_REF_341-R1A
CUFP_REF_361-R1A
FDP_REF_521-R1A
FP_REF_141-R1A
LUP_REF_421-R1A
NTP_REF_561-R1A
PP_REF_681-R1A

and 9 flow control valves:

FCV_541-R1A
FCV_570-R1A
FCV_571-R1A
FCV_574-R1A
FCV_622-R1A
FCV_642-R1A
FCV_662-R1A
FCV_688-R1A
FCV_690-R1A

Programs are designed modularly so they are all effectively identical.

Differentiating factors will be noted

An example of a final control element CM, a total of 17 such elements exist on the C300 corresponding to 8 pumps:

BMP_REF_241-R1A
CRP_REF_341-R1A
CUFP_REF_361-R1A
FDP_REF_521-R1A
FP_REF_141-R1A
LUP_REF_421-R1A
NTP_REF_561-R1A
PP_REF_681-R1A

and 9 flow control valves:

FCV_541-R1A
FCV_570-R1A
FCV_571-R1A
FCV_574-R1A
FCV_622-R1A
FCV_642-R1A
FCV_662-R1A
FCV_688-R1A
FCV_690-R1A

Programs are designed modularly so they are all effectively identical.

Differentiating factors will be noted

Figure 44: Final Control Element CM (FCV_570-R1A), Page 1.

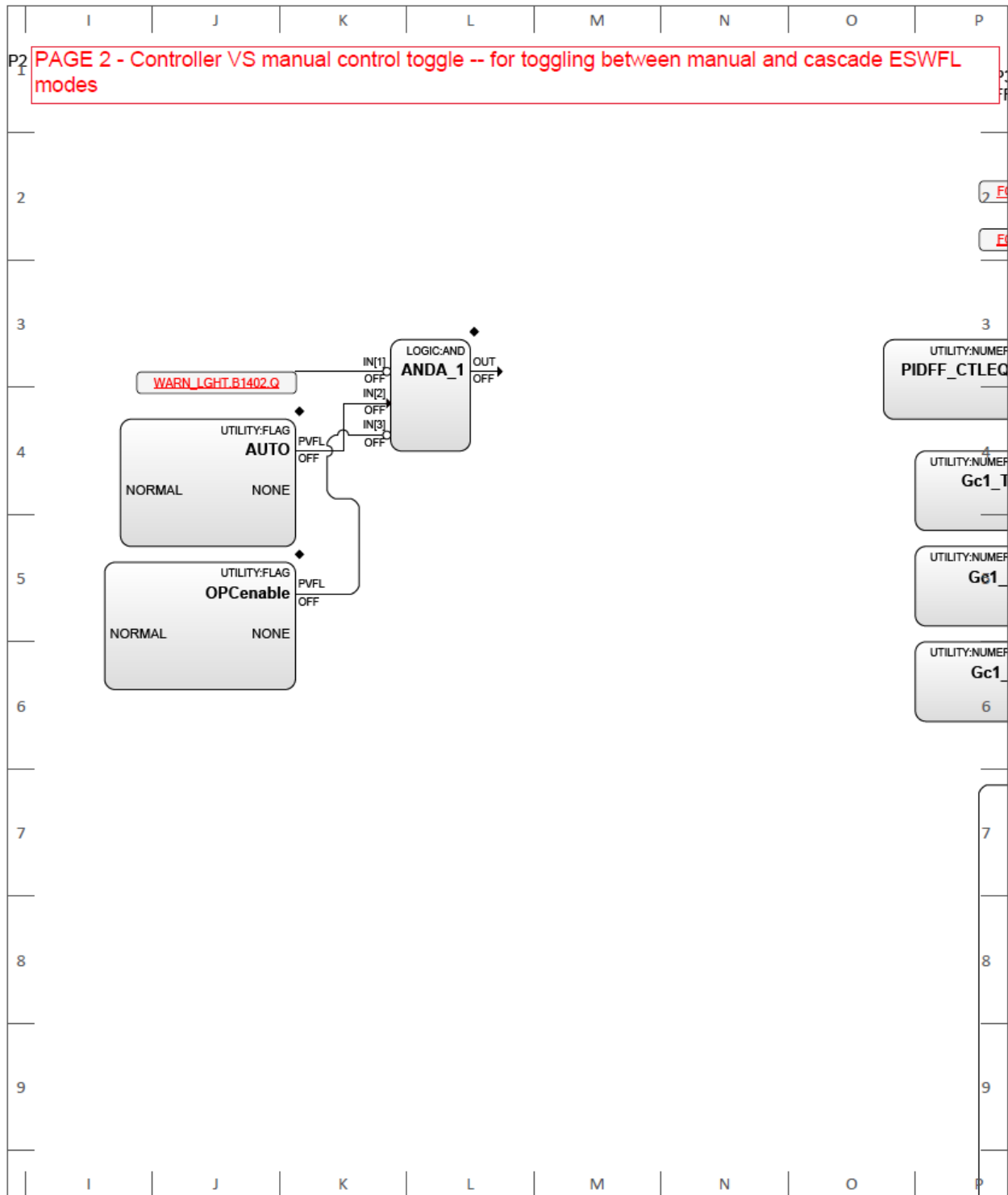


Figure 45: Final Control Element CM (FCV_570-R1A), Page 2.

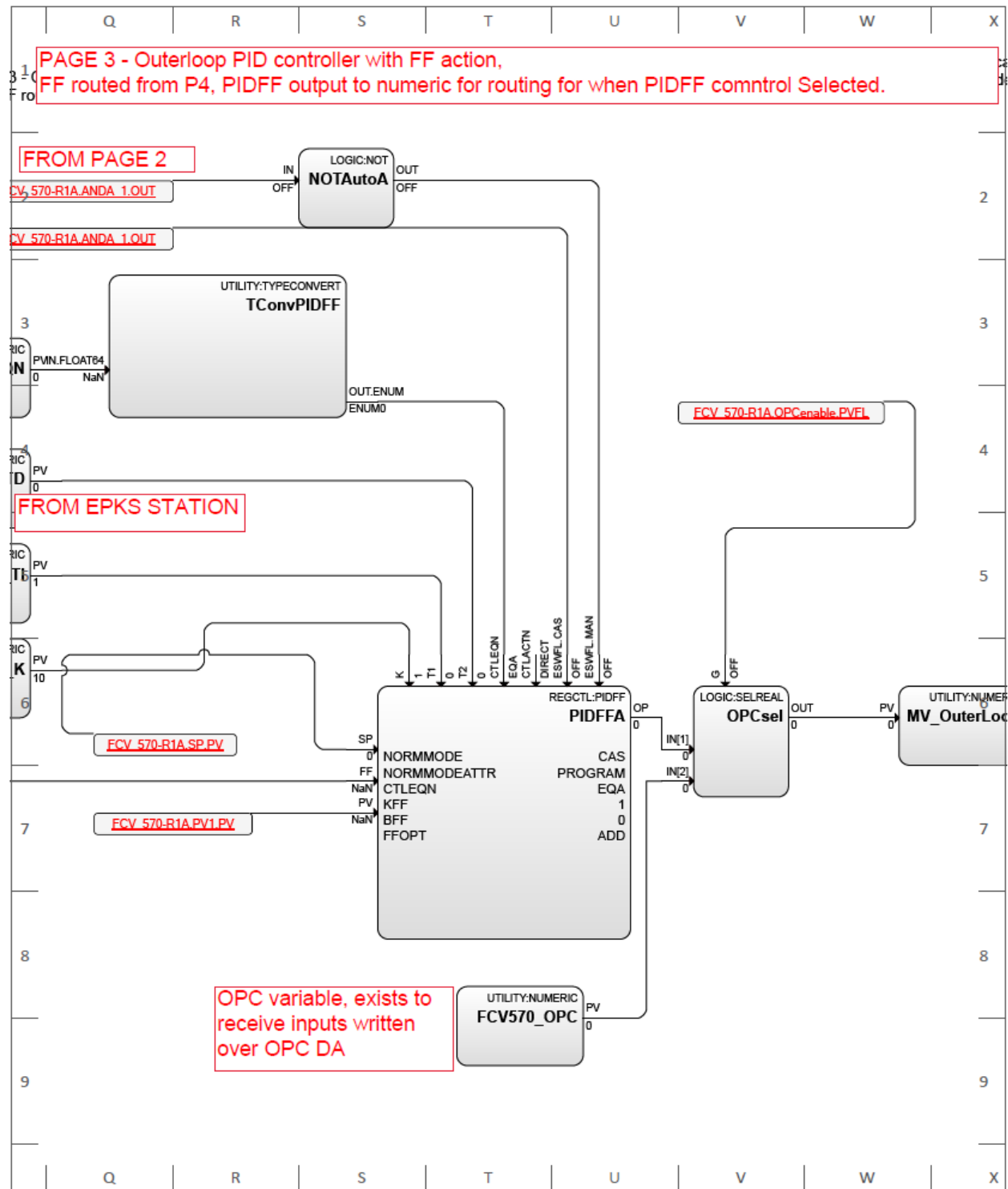


Figure 46: Final Control Element CM (FCV_570-R1A), Page 3.

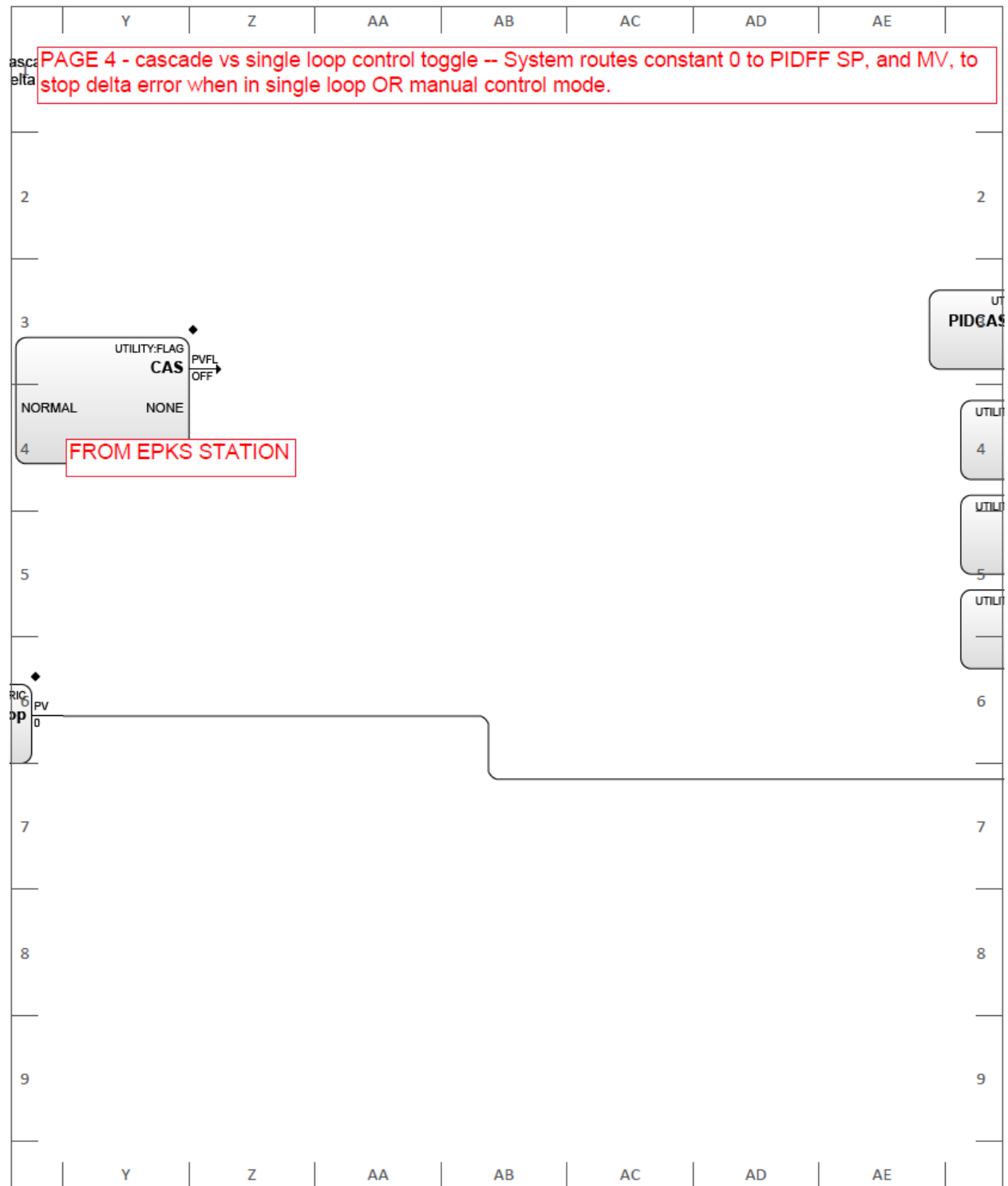


Figure 47: Final Control Element CM (FCV_570-R1A), Page 4.

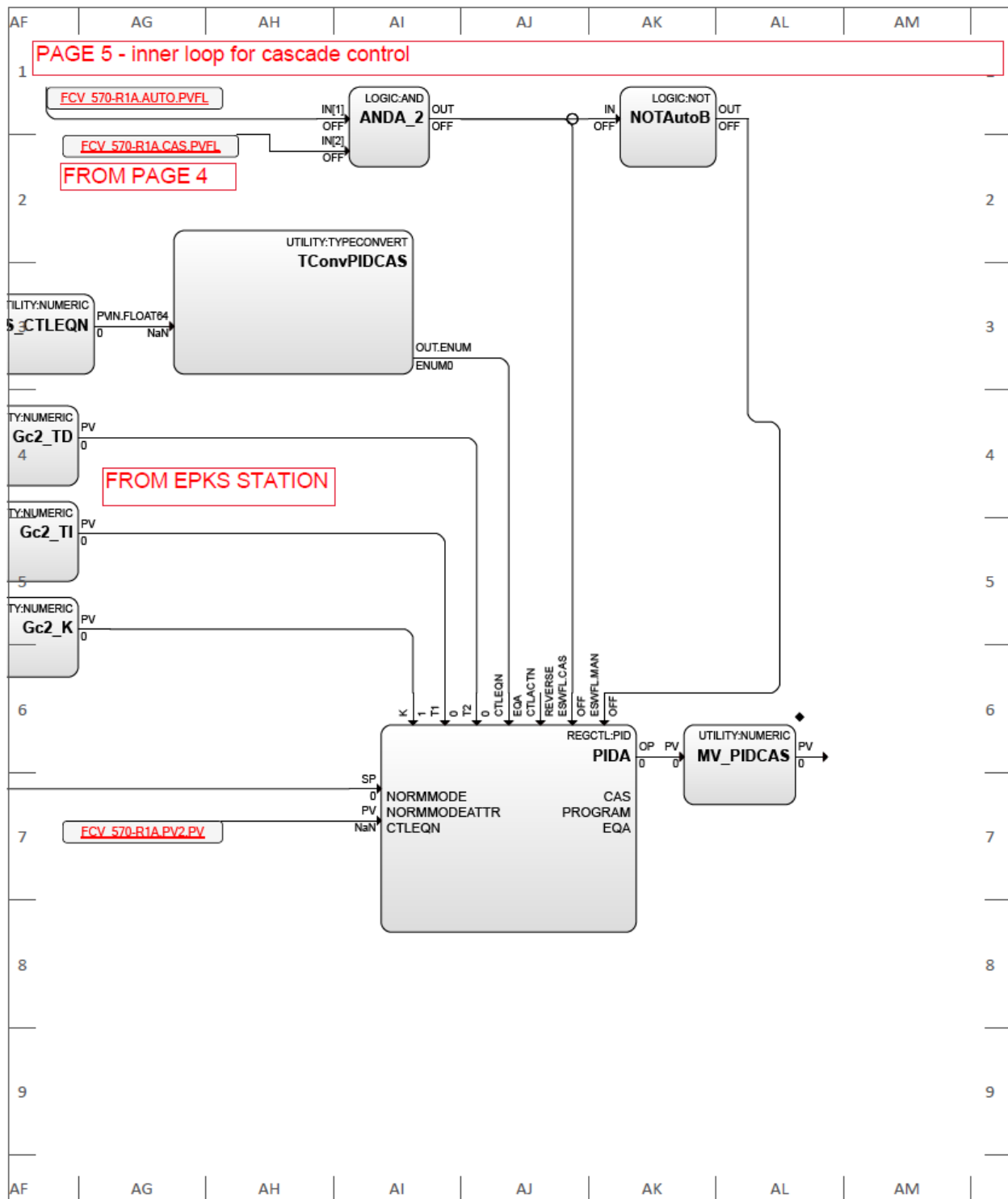


Figure 48: Final Control Element CM (FCV_570-R1A), Page 5.

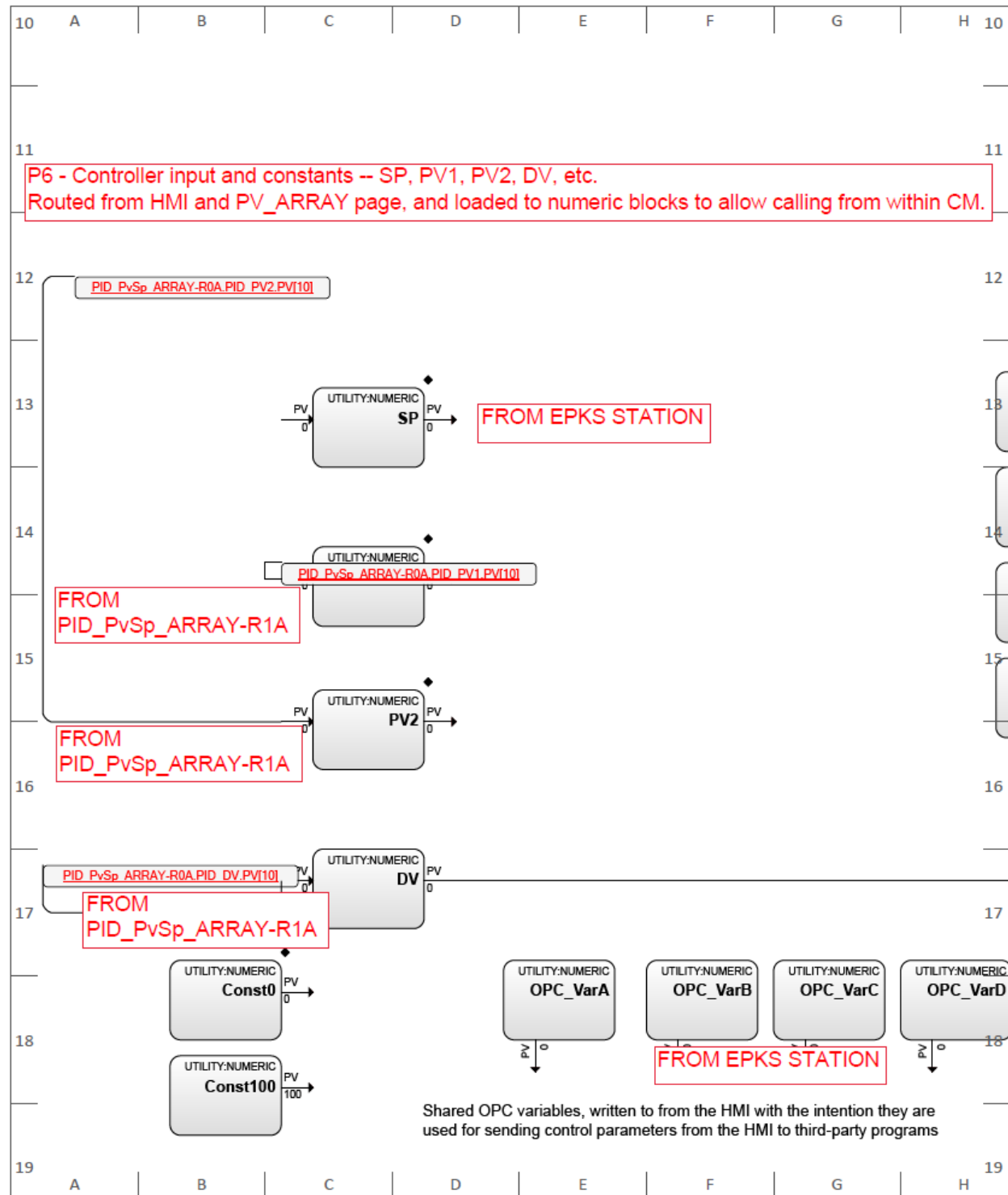


Figure 49: Final Control Element CM (FCV_570-R1A), Page 6.

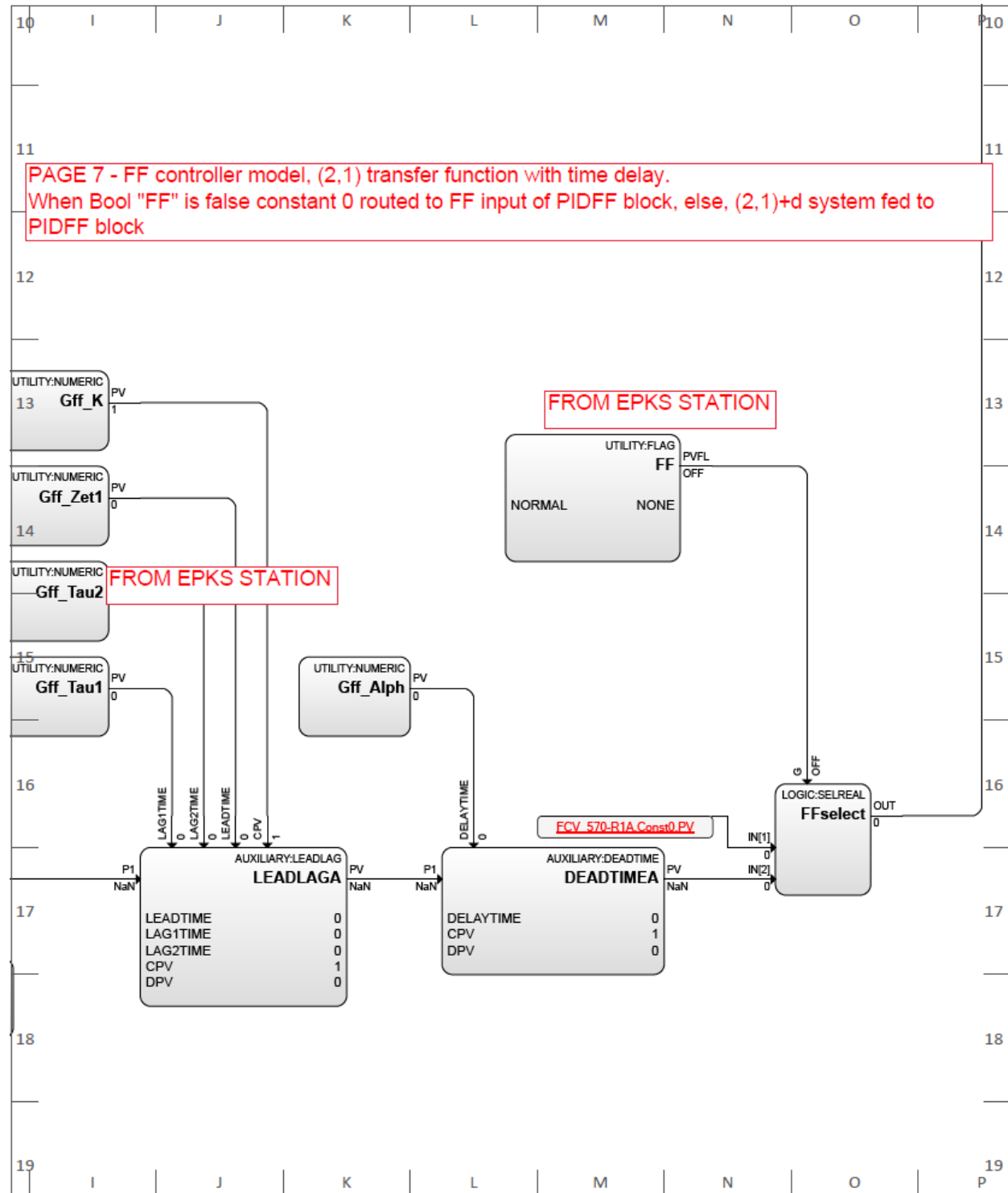


Figure 50: Final Control Element CM (FCV_570-R1A), Page 7.

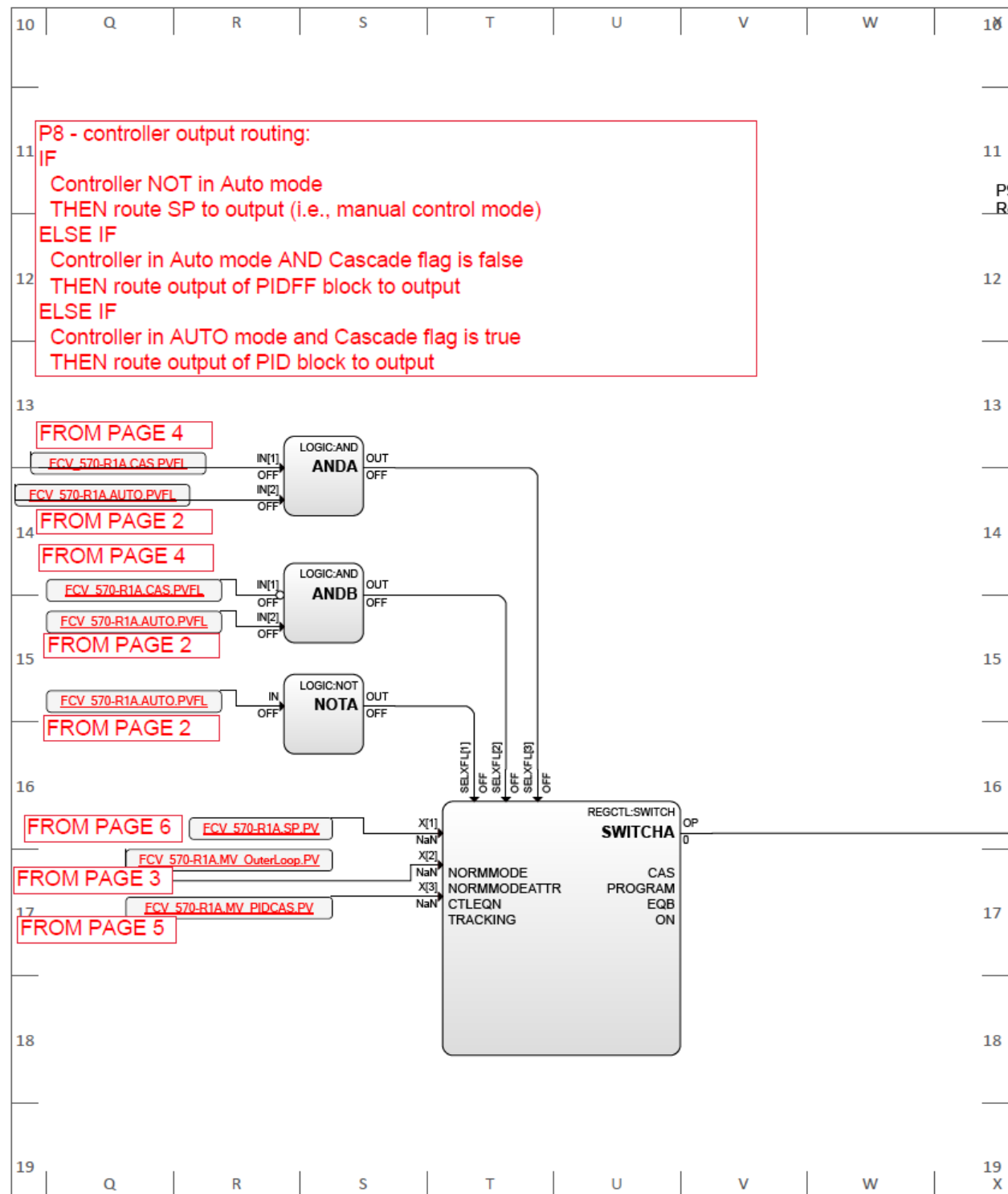
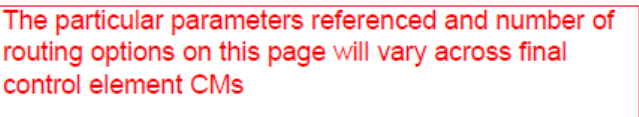


Figure 51: Final Control Element CM (FCV_570-R1A), Page 8.



64

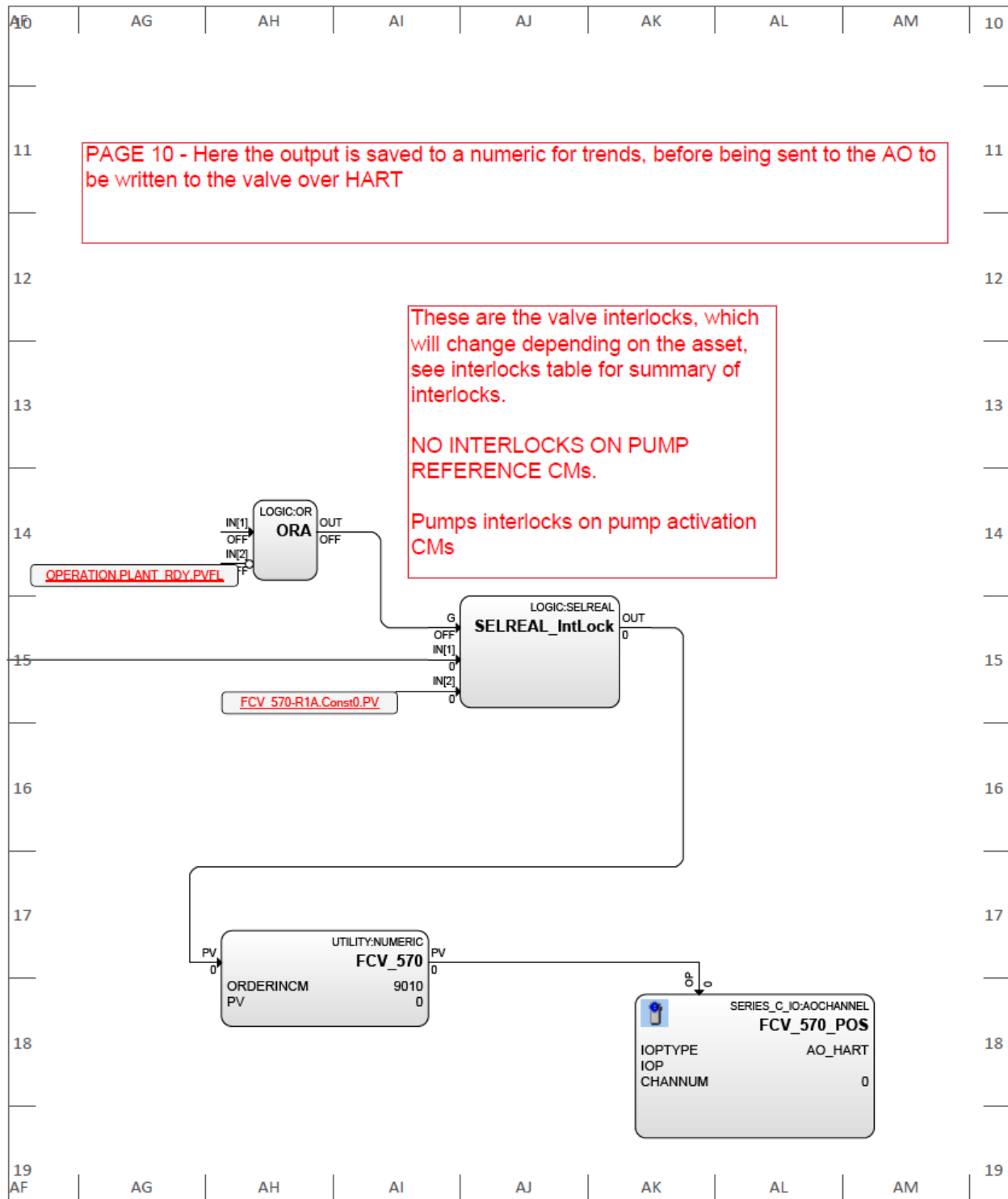


Figure 53: Final Control Element CM (FCV_570-R1A), Page 10.

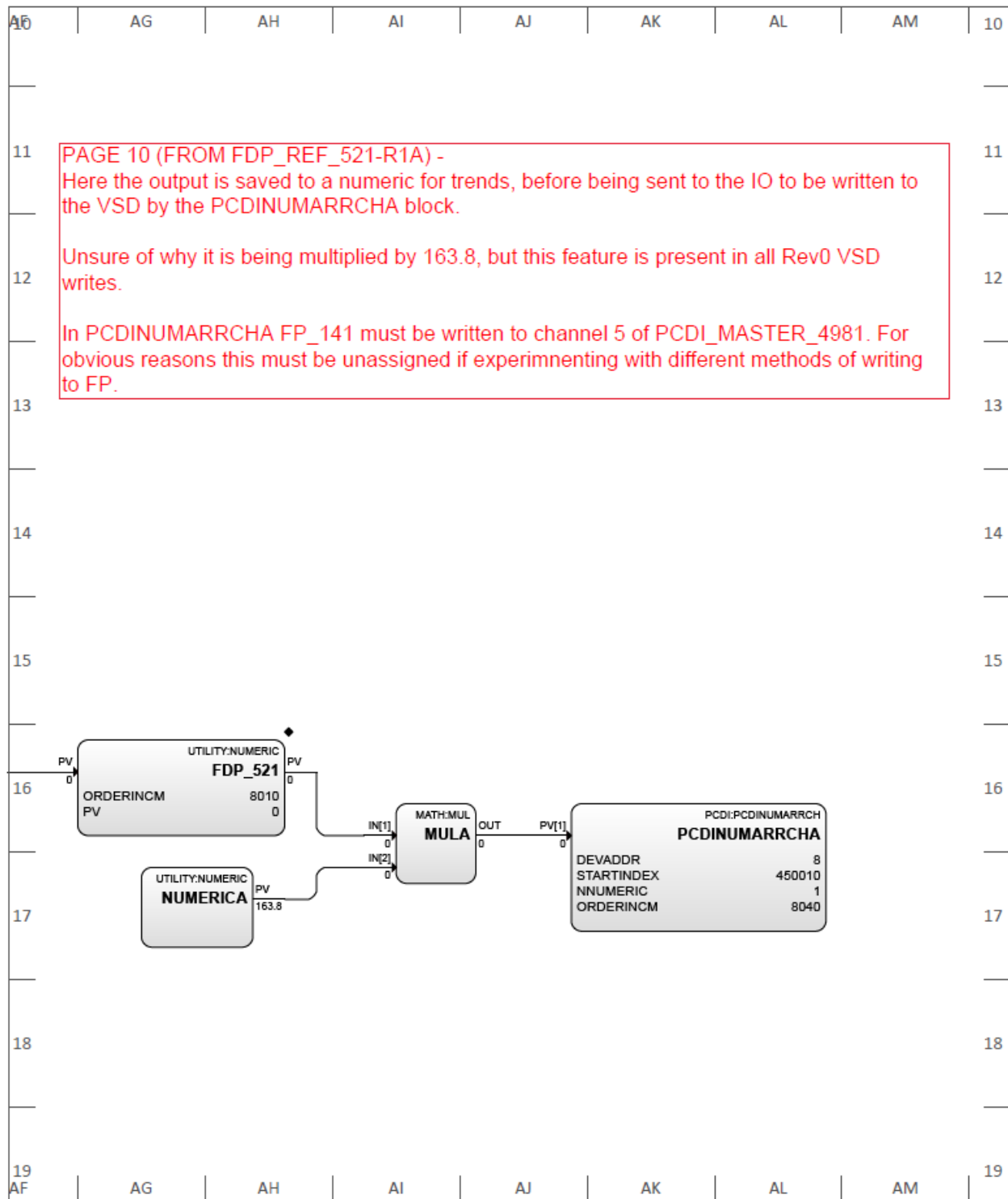


Figure 54: Final Control Element CM (FDP_REF_521-R1A), Page 10.